COMSATS University Islamabad, Sahiwal Campus

Medrotic

(AI-Powered Healthcare System)

A project presented to

COMSATS University Islamabad, Sahiwal Campus

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Muhammad Khaqan Nasir	**CIIT/FA22-BCS-039/SWL**
Muhammad Ibtisam Afzal	**CIIT/FA22-BCS-073/SWL**
Muhammad Adnan Tariq	**CIIT/FA22-BCS-082/SWL**

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted for any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Khaqan Nasir	**Muhammad Ibtisam Afzal**	**Muhammad Adnan Tariq**
----------	----------	----------

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) "Medrotic (AI-Powered Healthcare System)" was developed by **Muhammad Khaqan Nasir (CIIT/FA22-BCS-039/SWL)**, **Muhammad Ibtisam Afzal (CIIT/FA22-BCS-073/SWL)** and **Muhammad Adnan Tariq (CIIT/FA22-BCS-082/SWL)** under the supervision of **Dr. Muhammad Farhan** and that in his opinion it is fully adequate, in scope and quality for the degree of Bachelor of Science in Computer Sciences.

Supervisor
Dr. Muhammad Farhan
Assistant Professor
Department of Computer Science
COMSATS University Islamabad, Sahiwal Campus

Head of Department
Dr. Zafar Iqbal Roy
Assistant Professor
Department of Computer Science
COMSATS University Islamabad, Sahiwal Campus

Executive Summary

Medrotic was conceived as a practical response to real operational weaknesses found in traditional healthcare management environments, particularly in contexts where hospitals and clinics still depend on paper-based documentation or isolated digital tools. Such environments often suffer from slow access to medical history, duplicate patient records, repeated data entry, delayed interpretation of medical documents, and weak communication between stakeholders. The project aims to solve these problems through a centralized, secure, and intelligent healthcare platform that introduces automation into critical workflows without compromising doctor oversight and patient privacy.

The system provides three integrated dashboards. The Super Admin dashboard is designed for hospital-level management tasks such as doctor registration, verification, operational monitoring, and viewing institutional analytics. The Doctor dashboard supports clinical workflows through CNIC-based patient search, medical history access, prescription OCR, X-ray analysis, report review, and approval of AI-generated outcomes. The Patient dashboard gives end users-controlled access to their own medical data, allows them to add personal health information, upload reports, search for doctors, and interact with an AI-powered chatbot. This role-based separation ensures clarity of responsibility and controlled access to sensitive healthcare data.

The system was engineered using multi-tier architecture with Angular.js on the client side, ASP.NET Core on the application layer, Python FastAPI services for AI processing, and MS SQL Server on the data layer. The project also incorporates secure communication and notification mechanisms using WebSockets, SMTP, and SMS-oriented integrations mentioned in the design material. Development followed an Object-Oriented Design methodology together with an Iterative and Incremental process model, allowing the team to refine requirements and implementation over multiple cycles. Testing included manual, functional, integration, unit, and automated verification approaches. Overall, Medrotic demonstrates a complete final-year software project that integrates software engineering, databases, AI, security, testing, and web technologies into a domain-relevant product.

Acknowledgement

All praise is due to Almighty **Allah**, whose countless blessings, mercy, and guidance made it possible to complete this project successfully. The completion of Medrotic would not have been possible without the strength, patience, and understanding granted throughout the project journey, especially during the challenging phases of research, design refinement, implementation, and testing.

Sincere gratitude is extended to the project supervisor, **Dr. Muhammad Farhan**, for his valuable guidance, constructive feedback, and continued encouragement throughout the development of this project. His academic supervision played a central role in shaping the project direction, improving the system design, and ensuring that the work remained aligned with the objectives of a professional final year project. Gratitude is also extended to the faculty members and Final Year Project Committee of COMSATS University Islamabad, Sahiwal Campus, for their observations, suggestions, and academic support during the evaluation process.

Special thanks are also due to parents, family members, and friends whose moral support, prayers, and encouragement remained a source of motivation during the entire development cycle. Their trust and support helped sustain the effort required to bring this project from concept to implementation. Finally, appreciation is extended to all those individuals whose direct or indirect contribution helped in completing this academic work successfully.

Muhammad Khaqan Nasir	**Muhammad Ibtisam Afzal**	**Muhammad Adnan Tariq**
---	---	---

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CNIC	Computerized National Identity Card
CRUD	Create, Read, Update, Delete
CI/CD	Continuous Integration / Continuous Deployment
DFD	Data Flow Diagram
DL	Deep Learning
EHR	Electronic Health Record
FR	Functional Requirement
HCI	Human-Computer Interaction
HIPAA	Health Insurance Portability and Accountability Act
JWT	JSON Web Token
KPI	Key Performance Indicator
MFA	Multi-Factor Authentication
NFR	Non-Functional Requirement
OCR	Optical Character Recognition
OOD	Object-Oriented Design
OOP	Object-Oriented Programming
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
RAG	Retrieval-Augmented Generation
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language

Table of Contents

1 Introduction 2
1.1 Brief 2
1.2 Relevance to Course Modules 3
1.2.1 Software Engineering 3
1.2.2 Object Oriented Programming 3
1.2.3 Database Systems 4
1.2.4 Artificial Intelligence 4
1.2.5 Web Engineering 4
1.2.6 Data Structures and Algorithms 5
1.2.7 Computer Networks 5
1.2.9 Human Computer Interaction 6
1.2.10 Software Quality Assurance 6
1.3 Project Background 6
1.4 Literature Review 7
1.5 Analysis from Literature Review 8
1.6 Methodology and Software Lifecycle for This Project 9
1.6.1 Rationale Behind Selected Methodology 9
2 Problem Definition 11
2.1 Problem Statement 11
2.2 Deliverables and Development Requirements 11
2.2.1 User Interface (UI) Design 12
2.2.2 User Registration and Authentication 12
2.2.3 Super Admin Dashboard 12
2.2.4 Doctor Dashboard 12
2.2.5 Patient Dashboard 13
2.2.6 CNIC-Based Patient Identification 13
2.2.7 Medical History Management 13
2.2.8 Prescription Upload and OCR Module 13
2.2.9 X-ray Analysis Module 13
2.2.10 Test Report Analysis Module 14
2.2.11 AI Chatbot Integration 14
2.2.12 Doctor Search and Filtering 14
2.2.13 Analytics and Reporting 14
2.2.14 Notification System 14

2.2.15 Data Management and Storage 15
2.2.16 External API and Service Integration 15
2.2.17 Security and Privacy 15
2.2.18 Scalability and Performance Optimization 15
2.2.19 Web Application Development 16
2.2.20 Documentation 16
2.2.21 Testing and Quality Assurance 16
2.2.22 User Training and Support Materials 16
2.2.23 Maintenance, Updates and Continuous Improvement 16
2.3 Current System 17
3 Requirement Analysis 19
3.1 Use Case Diagrams 19
3.1.1 Use Case Diagram for Doctors 19
3.1.2 Use Case Diagram for Patients 20
3.1.3 Use Case Diagram for Admin 21
3.2 Detailed Use Case 22
3.3 Functional Requirements 29
3.4 Non-Functional Requirements 37
3.4.1 Usability 37
3.4.2 Performance 38
3.4.3 Security 38
3.4.4 Scalability 39
3.4.5 Reliability 40
3.4.6 Maintainability 40
3.4.7 Browser Compatibility 41
3.4.8 Communication Interfaces 41
4 Design and Architecture 44
4.1 System Architecture 44
4.2 Data Representation 45
4.3 Process Flow 48
4.4 Design Models 49
4.4.1 Class Diagram 50
4.4.2 Sequence Diagram for Doctor Dashboard 51
4.4.3 Sequence Diagram for Patient Dashboard 53
4.4.4 Sequence Diagram for Admin Dashboard 54
5 Implementation 56
5.1 Algorithm 56

5.2 External APIs 57
5.3 User Interface 58
5.3.1 Login Page 58
5.3.2 Signup Page 59
5.3.3 About Page 60
5.3.4 Contact Page 61
5.3.5 Home Page 62
5.3.6 FAQ Page 62
5.3.7 Doctor Dashboard 63
5.3.8 Prescription and X-ray Upload Pages 64
5.3.9 Patient Dashboard 65
5.3.10 Personal Metrics Page 65
5.3.11 Doctor Search Page 66
5.3.12 Chatbot Page 66
5.3.13 Admin Dashboard 67
5.3.14 Doctor Management Page 67
5.3.15 Doctor Verification Page 68
5.3.16 Analytics Page 68
5.3.17 UI Design Summary 69
6 Testing and Evaluation 71
6.1 Manual Testing 71
6.1.1 System Testing 71
6.1.2 Unit Testing 72
6.1.3 Functional Testing 74
6.1.4 Integration Testing 76
6.2 Automated Testing 77
7 Conclusion and Future Work 79
7.1 Conclusion 79
7.2 Future Work 80
7.2.1 Telemedicine Integration 80
7.2.2 Mobile Application Development 80
7.2.3 Advanced AI Capabilities 80
7.2.4 Pharmacy Integration and e-Prescriptions 81
7.2.5 Multi-Language and Regional Support 81
7.2.6 Enterprise Security Enhancements 81
7.2.7 Advanced Analytics Dashboard 81
References 82

List of Figures

Figure 3. 1: Use Case Diagram for Doctors 20
Figure 3. 2: Use Case Diagram for Patients 21
Figure 3. 3: Use Case Diagram for Admin 22
Figure 4. 1: System Architecture for Medrotic 45
Figure 4. 2: Data Representation Diagram of Medrotic 48
Figure 4. 3: Process Flow Representation of Medrotic 49
Figure 4. 4: Class Diagram of Medrotic 51
Figure 4. 5: Sequence Diagram of Doctor Dashboard 52
Figure 4. 6: Sequence Diagram of Patient Dashboard 53
Figure 4. 7: Sequence Diagram of Admin Dashboard 54
Figure 5. 1: Login Page 59
Figure 5. 2: Signup Page 59
Figure 5. 3: About Page 60
Figure 5. 4: Contact Page 61
Figure 5. 5: Home Page 62
Figure 5. 6: FAQs Page 63
Figure 5. 7: Doctor Dashboard 64
Figure 5. 8: Prescription Upload Page 64
Figure 5. 9: X-ray Upload Page 65
Figure 5. 10: Patient Dashboard 65
Figure 5. 11: Personal Metrics Page 66
Figure 5. 12: Doctor Search Page 66
Figure 5. 13: AI Chatbot Page 67
Figure 5. 14: Admin Dashboard Page 67
Figure 5. 15: Doctor Management Page 68
Figure 5. 16: User Verification Page 68
Figure 5. 17: Analytics Page 69

List of Tables

Table 1. 1: Literature Review 8
Table 3.2 1: Register Doctor 23
Table 3.2.2 Access Patient Data 23
Table 3.2.3 Upload Prescription 24
Table 3.2.4 Upload X-ray 24
Table 3.2.5 Patient Registration 25
Table 3.2.6 View Medical History 26
Table 3.2.7 Use AI Chatbot 26
Table 3.2.8 Search Doctors 27
Table 3.2.9 View Analytics 27
Table 3.2.10 Send Notifications 28
Table 3.2.11 Add Personal Metrics 28
Table 3.2.12 Test Report Analysis 29
Table 3.3. 1: Doctor Registration 30
Table 3.3. 2: Patient Access via CNIC 30
Table 3.3. 3: Patient Registration 31
Table 3.3. 4: Prescription Upload and OCR 31
Table 3.3. 5: X-ray Upload and Analysis 32
Table 3.3. 6: View Medical History 33
Table 3.3. 7: Add Personal Medical History 33
Table 3.3. 8: AI Chatbot for Queries 34
Table 3.3. 9: Test Report Analysis 34
Table 3.3. 10: Doctor Search 35
Table 3.3. 11: View Hospital Analytics 36
Table 3.3. 12: Notification Delivery 36
Table 3.4. 1 Usability 37
Table 3.4. 2 Performance 38
Table 3.4. 3 Security 39
Table 3.4. 4 Scalability 39
Table 3.4. 5 Reliability 40
Table 3.4. 6 Maintainability 41
Table 3.4. 7 Browser Compatibility 41
Table 3.4. 8 Communication Interfaces 42
Table 5.2. 1: Details of APIs used in the Project 57
Table 6. 1: System Testing Results 71
Table 6. 2: Doctor Login and Patient Search 72
Table 6. 3: Prescription Upload and OCR 73
Table 6. 4:Patient Self-Registration 74
Table 6. 5: Doctor Module Functional Testing 74
Table 6. 6: Patient Module Functional Testing 75
Table 6. 7: Super Admin Module Functional Testing 76
Table 6. 8: Integration Testing Results 76
Table 6. 9: Automated Testing Tools and Coverage 77

Chapter 1
Introduction

1 Introduction

Medrotic is a modern web-based healthcare platform designed to streamline interactions among hospitals, doctors, and patients by bringing data management, clinical workflows, and diagnostic support into one secure environment. The system addresses long-standing inefficiencies in traditional healthcare operations by replacing fragmented and manual processes with centralized digital services that support both administrative and clinical needs.

Instead of using disconnected systems for registration, record management, diagnostics, and communication, Medrotic integrates all these tasks into a unified platform supported by role-specific dashboards and AI-enabled automation.

The project is especially relevant in healthcare settings where institutions require affordable digital transformation but face barriers in adopting expensive enterprise systems **[1]**. Medrotic was designed to serve small and medium-sized providers by offering enterprise-style features such as secure record handling, analytics, automated document interpretation, and patient self-service tools through open and modular technologies.

The report documents the full development of the system, including its academic relevance, problem context, literature foundations **[Table 1.1]**, requirements, design, implementation, and testing.

1.1 Brief

Medrotic is an AI-powered web-based healthcare management system designed to improve the interaction between hospitals, doctors, and patients through secure, centralized, and intelligent automation.

The platform supports role-based dashboards for Super Admin, Doctor, and Patient, enabling efficient medical record management, diagnostic support, and healthcare communication. It uses CNIC as the primary patient identifier, which helps reduce duplicate records and improves the speed and accuracy of patient lookup and record retrieval.

The system integrates AI technologies for prescription OCR **[Table 3.2.3]**, deep learning-based image analysis, and chatbot-assisted patient queries **[Table 3.2.7]**. Medrotic is built using Angular.js for the frontend, ASP.NET Core for backend services, Python for AI processing, and MS SQL Server for data

storage. The overall goal of the project is to provide a practical, scalable, and secure digital healthcare solution for small to medium-sized healthcare providers.

1.2 Relevance to Course Modules

The development of Medrotic integrates various concepts and skills gained throughout the Computer Science degree program.

1.2.1 Software Engineering

Software Engineering played a foundational role in the development of Medrotic because the entire project followed a structured software development process beginning from problem identification and requirement gathering and extending to design, implementation, testing, and documentation. Concepts such as requirement elicitation, feasibility awareness, lifecycle planning, traceability, use case analysis **[Fig 3.1, Fig 3.2, Fig 3.3]**, and validation were directly applied during the preparation of the Scope Document, SRS, SDD, and Project Report. The use of an Iterative and Incremental model supported by Agile practices also reflects the application of software engineering principles in managing changing requirements and multiple development cycles **[1]**.

This course was also relevant in terms of quality assurance and stakeholder communication. The identification of actors, use cases, dependencies, and constraints came from software engineering techniques commonly taught in the BSCS curriculum. In the context of Medrotic, this knowledge ensured that the system remained aligned with stakeholder needs while maintaining consistency across all design and implementation artifacts.

1.2.2 Object Oriented Programming

Object-Oriented Programming was central to the conceptual and structural design of Medrotic. The system naturally consists of interacting entities such as User, Patient, Doctor, Super Admin, MedicalRecord, Document, AIResult, and Notification. These entities map cleanly to object-oriented classes and relationships, allowing the project to be designed in a modular, reusable, and maintainable manner. Inheritance is particularly evident in the role hierarchy where Patient, Doctor, and Super Admin are specialized forms of a common User entity.

Encapsulation and abstraction also support the development of separate modules with controlled responsibilities, while polymorphic behavior can be applied where role-based operations differ under shared structures. The project demonstrates practical application of OOP concepts beyond classroom exercises by using them in the architecture, database mapping strategy, and service interaction patterns of a complete software product.

1.2.3 Database Systems

Database Systems knowledge was used extensively in Medrotic for designing the relational schema, defining entities and relationships, enforcing integrity constraints, and ensuring efficient retrieval of medical records. The project includes structured tables for users, patients, doctors, super admins, medical records, documents, AI results, and notifications. The use of indexing, normalization, primary and foreign keys, and inheritance mapping through TPH demonstrates the direct influence of database design concepts on the project.

This course was also relevant in implementing search optimization and secure persistence of sensitive healthcare data. CNIC-based patient access requires fast and reliable querying, while document and AI result storage require traceable relations between uploaded files and clinical outputs. As a result, Medrotic gave the team hands-on experience in transforming an object-oriented model into a robust relational design suitable for real-world applications.

1.2.4 Artificial Intelligence

Artificial Intelligence is one of the most distinctive components of Medrotic. The project incorporates OCR for extracting content from uploaded prescriptions and deep learning techniques for analyzing X-rays and test reports. These features convert static files into machine-readable, clinically relevant outputs that assist both doctors and patients. The chatbot functionality further extends AI relevance by enabling natural interaction for health-related queries linked to patient context.

From an academic perspective, this project applied AI concepts such as model inference, confidence handling, supervised diagnostic assistance, and response generation within a real system. The use of AI in Medrotic is not theoretical only; it directly affects workflows, approval logic, user experience, and automation of previously manual tasks. This made the course highly relevant to both the conceptual motivation and the technical implementation of the project.

1.2.5 Web Engineering

Web Engineering is reflected in the planning and development of Medrotic as a responsive multi-role web platform. The project required the design of role-specific interfaces, form validation, client-side interactions, page routing, service calls, and workflow-driven navigation across dashboards. Since the system is intended to operate through browsers for different types of users, web engineering concepts were essential in ensuring usability, maintainability, and structured frontend behavior.

The course was also relevant in applying layered development principles and separating concerns between frontend and backend logic. Practical work in Medrotic demonstrates how a complex domain

system can be translated into an accessible browser-based solution with clear interfaces and controlled access, which is one of the primary outcomes expected from Web Engineering education.

1.2.6 Data Structures and Algorithms

Although Medrotic is not a purely algorithmic system, Data Structures and Algorithms still contributed to the project in meaningful ways. Efficient searching, filtering, input validation, document classification routing, analytics aggregation, and record ordering all depend on careful thinking about data organization and procedural logic. Patient lookup using CNIC, chronological history arrangement, and notification dispatch mechanisms all involve algorithmic reasoning.

The implementation chapter of the design materials also identifies core algorithms such as patient identification, document classification, AI analysis handling, approval workflows, medical record update routines, and analytics caching. These reflect the practical application of algorithmic design in building efficient and reliable software that works correctly under real operational conditions.

1.2.7 Computer Networks

Computer Networks is relevant because Medrotic depends on reliable communication among multiple layers and services. The frontend communicates with backend APIs, the backend exchanges data with AI services, and external services such as notifications and real-time channels support communication across the system. Concepts such as client-server communication, HTTP-based service exchange, WebSockets, API endpoints, and distributed deployment are all directly tied to networking principles. This relevance becomes stronger in a healthcare environment where timely and secure transfer of information is essential. Notifications, analytics refreshes, chatbot interactions, and API integrations all require understanding of network-based communication and service interoperability, making the course clearly connected to the design and implementation of the project.

Information Security is critically important in Medrotic because the system stores and processes highly sensitive patient data. The project applies security measures such as role-based access control, secure authentication, encrypted communication, audit logging, and secure handling of identity information. These measures are consistent with the security expectations stated in the SRS and design materials.

The course contributed to the understanding of confidentiality, integrity, access restriction, and accountability in software systems. In Medrotic, those principles are translated into practical implementation decisions including HTTPS-based communication, JWT or token-based authentication flows, authorization boundaries for different roles, and logging of important changes. This makes the project a strong example of applied secure software development.

1.2.9 Human Computer Interaction

Human-Computer Interaction contributed to Medrotic through the design of interfaces that are understandable, efficient, and role-appropriate. Healthcare systems are used by people with different skill levels and responsibilities, so interface clarity is essential. Medrotic addresses this through clean dashboards, intuitive flows for registration and uploads, progressive interaction for approvals, and simple forms for patient-facing features.

The usability expectations documented in the SRS, such as enabling new users to complete registration and first login within a short time, reflect direct HCI concerns. This course helped in shaping interface organization, error messaging, accessibility, workflow visibility, and trust-building through consistent interaction design across the platform **[Fig 5.1 - 5.17]**.

1.2.10 Software Quality Assurance

Software Quality Assurance influenced the testing strategy, requirement verification, and quality-focused mindset adopted during development. Medrotic required manual testing, unit testing, integration testing, functional testing, and automated testing to ensure correctness across diverse modules. This course provided the conceptual basis for planning how features should be validated and how test coverage contributes to software reliability.

The project also benefited from SQA in terms of traceability and defect prevention. Since healthcare software cannot tolerate careless inconsistencies, the quality assurance perspective helped ensure that requirements defined in the SRS were preserved in the SDD and implemented systematically in the application. This strengthens the professional quality of the overall project.

1.3 Project Background

The need for Medrotic emerged from the observation that many healthcare institutions still rely on manual or partially digitized workflows for patient registration, document handling, and record maintenance. In such settings, patient data is often fragmented across files, departments, or separate systems, making it difficult to retrieve complete histories when needed. Doctors may spend valuable time locating or reconstructing patient information, while patients often remain dependent on institutions for access to their own records.

At the same time, diagnostic documents such as prescriptions, test reports, and X-rays are often processed manually, which increases administrative workload and creates opportunities for transcription errors or delays. Existing enterprise systems can solve some of these issues, but their cost, complexity, and vendor dependency make them less practical for smaller institutions. Medrotic was

therefore developed as an open, modular, and AI-supported platform that can modernize healthcare operations while remaining accessible, scalable, and academically grounded.

1.4 Literature Review

The healthcare technology landscape includes a variety of hospital management systems, electronic health record solutions, cloud-based health APIs, and appointment-oriented patient platforms. These systems demonstrate that digital transformation in healthcare can significantly improve record organization, communication, and decision support. However, the literature and related systems also show that most existing solutions are either highly specialized, expensive, cloud-dependent, or limited in terms of integrated AI-driven workflows **[1]**.

Enterprise systems such as Epic Systems and Cerner are known for broad hospital information management capabilities. They support clinical workflows, patient administration, and data exchange at scale. However, such systems are generally associated with high deployment cost, vendor lock-in, and limited flexibility for customization by small and medium-sized healthcare providers. Their closed ecosystems also make experimentation and educational deployment difficult, especially in contexts where local adaptation is necessary.

Cloud-oriented platforms such as Google Cloud Healthcare API demonstrate the growing importance of scalable medical data processing and AI-assisted health services. These solutions are strong in interoperability, cloud integration, and service extensibility. Nevertheless, the project documents indicate that such platforms do not provide a unified complete role-based dashboard system by themselves and often require continued cloud dependency. For institutions seeking a self-managed and directly usable healthcare application, this creates both operational and financial constraints.

Consumer-facing services such as Practo focus more on doctor discovery, appointment support, and patient engagement features. While these platforms improve accessibility and convenience for end users, they often do not integrate deeper AI-based diagnostic support, prescription OCR, or internal hospital analytics in the way required by Medrotic. Their feature set is useful for comparison because it highlights patient-side service expectations, but it also exposes the need for stronger clinical and administrative integration **[2]**.

Beyond named systems, many generic local hospital software products remain fragmented in their functionality. Some focus primarily on billing, others on basic registration, and others on limited record management. Such systems often lack advanced automation, consistent patient identity handling, real-time analytics, and integrated workflows across hospitals, doctors, and patients. As a result, literature

and comparative system analysis both support the need for a unified system that combines operational management, AI support, and user empowerment.

Table 1. 1: Literature Review

System Name	Strengths	Weaknesses	Relevance to Medrotic
Epic Systems	Comprehensive healthcare management, trusted enterprise solution	High cost, closed source	Shows the need for a more affordable and flexible solution
Cerner	Strong clinical and hospital management capabilities	Complex deployment, expensive licensing	Reinforces the value of a simpler open-source platform
Google Cloud Healthcare API	Powerful cloud integration and AI support	Cloud dependency, lacks unified dashboards	Supports the idea of integrated role-based dashboards
Practo	Easy doctor discovery and appointment support	Limited AI and diagnostic automation	Highlights the need for OCR and deep learning in healthcare
Traditional Local Hospital Software	Familiar workflows, localized usage	Fragmented records, limited automation, weak analytics	Need for centralized AI-enabled workflow platform

The literature review therefore establishes three major insights. First, healthcare systems must be integrated across user roles rather than optimized for only one group. Second, AI support is increasingly valuable in automating medical data interpretation and reducing manual effort. Third, affordability, flexibility, and modularity remain essential if digital healthcare systems are to be viable for smaller institutions. Medrotic was designed directly in response to these insights, combining centralized record management, AI processing, role-based dashboards, and open technology choices in a single platform.

1.5 Analysis from Literature Review

The literature review clearly indicates that the healthcare software ecosystem is rich in solutions but uneven in accessibility, integration, and intelligence. Some platforms are operationally powerful but financially inaccessible, while others are convenient for patients but limited in clinical workflow support. The gap that consistently appears is the absence of a low-cost, AI-assisted, role-integrated platform that can be deployed in medium-scale settings without dependence on expensive proprietary ecosystems.

Medrotic responds to this gap by unifying the needs of administrators, doctors, and patients into one secure platform. The adoption of CNIC as a central identity mechanism reflects the need for unambiguous patient linkage. The integration of OCR and deep learning reflects the importance of automating document-heavy and image-heavy tasks. The inclusion of analytics, notifications, and self-service features addresses the broader literature insight that healthcare systems should not merely store data but should actively support decision-making and patient engagement.

1.6 Methodology and Software Lifecycle for This Project

Medrotic was developed using an Object-Oriented Design approach combined with an Iterative and Incremental software process model with Agile practices. This combination was selected because the system contains multiple interacting modules, AI components, and role-based workflows that benefit from gradual development and continuous refinement. Each increment of the project focused on a specific set of requirements and was validated before moving to the next phase.

The Agile-based approach allowed the team to gather feedback, update requirements, and improve the system step by step. This was especially important for the AI modules, where model accuracy, workflow approval logic, and integration with medical records required repeated testing and adjustment. The methodology also helped the team manage complexity while maintaining traceability between SRS, SDD, and implementation [3].

1.6.1 Rationale Behind Selected Methodology

Object-Oriented Design was appropriate because Medrotic is built around real-world entities such as Patient, Doctor, Super Admin, MedicalRecord, Document, and AIResult. These entities naturally map to classes and relationships, which makes the system easier to understand, extend, and maintain.

The Iterative and Incremental model was chosen because the project required continuous delivery and testing of features like authentication, patient lookup, file upload, AI analysis, and dashboard management. Agile practices supported regular review and refinement, which was important for a healthcare system where correctness, usability, and security are essential **[5]**.

Chapter 2
Problem Definition

2 Problem Definition

The healthcare sector demands timely access to accurate patient information, reliable communication among stakeholders, and efficient handling of clinical documents. In many real settings, however, medical workflows remain fragmented due to paper-based records [3], isolated software tools, or partial digitization. These limitations affect administrative efficiency, diagnostic speed, and patient satisfaction. Medrotic was proposed as a targeted solution to this problem by building a centralized platform that automates routine work while maintaining human oversight where clinical approval is required.

2.1 Problem Statement

Healthcare institutions continue to face serious operational problems caused by fragmented records, repeated manual data entry, slow document processing, and limited patient visibility into their own healthcare history. Doctors often need fast access to a patient's complete record but must instead navigate incomplete files or isolated sources of information. Patients may be unable to retrieve their own history easily, and hospital administrators may not have clear real-time insight into institutional activity.

These problems are intensified by the fact that many existing digital solutions are either incomplete or too costly to adopt. Proprietary systems may provide advanced features, but their pricing, deployment complexity, and vendor dependency create barriers for smaller providers. At the same time, healthcare workflows increasingly involve images, prescriptions, reports, and communication events that can benefit from automation. Medrotic addresses this problem by offering a secure, AI-powered, role-based platform that combines centralized record management with practical automation and analytics

2.2 Deliverables and Development Requirements

The Medrotic project includes a complete set of development requirements and deliverables to ensure the successful implementation of a secure, intelligent, scalable, and user-friendly AI-powered healthcare platform. These deliverables cover user interface design, role-based dashboards, patient and doctor workflows, AI-assisted healthcare features, backend services, data management, security, notifications, testing, documentation, and long-term maintenance. Since Medrotic is intended to serve hospitals, doctors, and patients through one integrated ecosystem, the development requirements focus not only on functionality but also on reliability, privacy, usability, and future extensibility. The final deliverable is a fully functional web-based healthcare management system that supports secure CNIC-

based patient identification, AI-driven diagnostic assistance, real-time analytics, and seamless interaction among all stakeholders.

2.2.1 User Interface (UI) Design

Design an intuitive, responsive, and role-specific web interface that allows Super Admins, Doctors, and Patients to navigate the system efficiently and comfortably. The user interface must present clean layouts, meaningful dashboards, simple forms, upload panels, medical history views, and clear action controls so that each user can complete tasks without confusion. Since Medrotic is a healthcare platform, the interface must emphasize readability, trust, accessibility, and simplicity while remaining compatible with modern browsers and different screen sizes.

2.2.2 User Registration and Authentication

Implement a secure authentication and authorization mechanism for all system users using role-based access control. Patients should be able to register themselves using CNIC, while doctors should be registered and verified through the Super Admin. The system must ensure that only authenticated users can access their relevant dashboards, and every role should be limited to the functions assigned to it, such as patient self-service, doctor-level clinical access, or hospital-level administrative control.

2.2.3 Super Admin Dashboard

Develop a dedicated Super Admin dashboard that enables hospital-level management and system governance. The dashboard should allow administrators to register and verify doctors, manage hospital-level information, monitor user growth, and view real-time analytics such as patient counts, doctor counts, and overall activity indicators. This module acts as the control center of the platform and ensures that administrative users maintain oversight of the complete healthcare environment.

2.2.4 Doctor Dashboard

Develop a specialized Doctor dashboard that supports clinical workflows in an efficient and secure manner. Doctors should be able to log in, search patients through CNIC, access and update patient data, upload prescriptions for OCR processing, upload X-rays for AI-based analysis, review AI-generated findings, and approve critical outputs before they are stored in the medical record. The dashboard must reduce manual work while preserving doctor authority over clinically sensitive decisions.

It should also include role-based access control, audit trails, and data encryption to ensure compliance with healthcare privacy and security standards.

2.2.5 Patient Dashboard

Develop a patient-facing dashboard that empowers individuals to manage and review their own health information. Patients should be able to self-register, log in securely, view their medical history in chronological order, add personal health metrics, upload reports for analysis, interact with the AI chatbot, and search for doctors by specialty, city, disease, or experience. The dashboard should be simple, engaging, and accessible so that even non-technical users can benefit from the system with ease.

2.2.6 CNIC-Based Patient Identification

Implement a secure patient identification and retrieval mechanism using CNIC as the primary unique key. This feature must enable fast and accurate patient lookup while preventing duplicate records and maintaining continuity across medical workflows. Since Medrotic uses CNIC as the foundation for linking patient data, the system must validate CNIC format and ensure that only authorized users can retrieve related medical information.

2.2.7 Medical History Management

Provide complete medical history management features that allow healthcare records to be stored, retrieved, and presented in chronological order. Doctors should be able to append new diagnoses, treatment notes, and AI-assisted outcomes, while patients should be able to view their own historical records and contribute personal health metrics where allowed. This module should ensure that medical records remain organized, traceable, and easy to interpret across time.

2.2.8 Prescription Upload and OCR Module

Implement a prescription processing module where doctors can upload prescription images and the system can automatically extract textual information using OCR models. The extracted content should then be associated with the patient's medical record, reducing the need for manual transcription. This deliverable is important for digitizing paper-based or image-based prescriptions and improving efficiency in clinical documentation workflows.

2.2.9 X-ray Analysis Module

Develop an AI-powered X-ray analysis module that allows doctors to upload X-ray images for automated interpretation through deep learning models. The system should analyze the uploaded image, produce findings or insights, and place the result in a review state until approved by the

responsible doctor. This module must balance automation with professional oversight so that AI supports clinical decision-making without replacing it.

2.2.10 Test Report Analysis Module

Implement a test report analysis module that enables users to upload supported medical reports and receive AI-generated insights. When reports are uploaded through doctor-led workflows, the system should preserve approval and review requirements where necessary. This module expands Medrotic beyond prescriptions and X-rays by supporting broader document-based medical workflows and helping convert raw uploaded files into more useful digital information.

2.2.11 AI Chatbot Integration

Develop an AI chatbot integrated with the patient-side environment to answer health-related queries and provide guided assistance based on available system context. The chatbot should support better patient engagement, improve accessibility to information, and make the platform more interactive. It should serve as an intelligent support feature rather than a replacement for professional diagnosis, helping users navigate their data and common health-related concerns more easily.

2.2.12 Doctor Search and Filtering

Provide a doctor discovery feature through which patients can search for doctors using criteria such as city, specialty, disease area, or experience. The system must offer efficient search and filtering mechanisms so that users can quickly locate relevant doctors according to their needs. This feature enhances patient convenience and improves the usefulness of the platform as a complete healthcare interaction system rather than only a record repository.

2.2.13 Analytics and Reporting

Develop an analytics module that aggregates and displays important hospital-level KPIs in real time. Super Admins should be able to view operational insights such as total doctors, total patients, and other performance indicators through visual dashboards. The analytics module should support informed decision-making, resource planning, and system monitoring while respecting privacy rules such as anonymization where required.

2.2.14 Notification System

Implement a multi-channel notification system that keeps users informed about important events such as approvals, updates, analysis completion, and system activity. Notifications should be delivered

through supported channels such as dashboard alerts, email, SMS, or real-time updates where configured. This module is necessary for maintaining responsiveness across workflows and ensuring that patients, doctors, and administrators remain aware of changes relevant to them.

2.2.15 Data Management and Storage

Develop a robust data management layer responsible for storing and retrieving users, medical records, uploaded documents, AI outputs, and notifications securely and efficiently. The database design must support indexing, structured relationships, data integrity, and scalable storage of healthcare records. It should also support chronological retrieval of records, role-based access, and efficient query performance for frequently used operations such as CNIC-based patient lookup.

2.2.16 External API and Service Integration

Provide support for internal and external APIs to enable AI processing, notifications, and future third-party integration. FastAPI-based AI services should handle OCR and deep learning inference, while communication services such as SMTP, SMS gateways, and WebSockets should support notifications and real-time updates. The system architecture should also remain open for integration with future EHR, telehealth, laboratory, or healthcare support systems where expansion is needed.

2.2.17 Security and Privacy

Prioritize the protection of sensitive healthcare data through strong security controls and privacy-aware design. The system must implement HTTPS-based secure communication, role-based access control, secure handling of CNIC and user credentials, audit logging, and restricted access to medical data according to role. Since Medrotic processes confidential healthcare information, security is a core development requirement and not an optional enhancement.

2.2.18 Scalability and Performance Optimization

Ensure that the system remains responsive, stable, and scalable as usage grows. Medrotic should be able to support increasing numbers of patients, doctors, records, uploads, and concurrent users without significant performance degradation. The architecture should support modular scaling through containers and service separation, while query optimization and efficient API design should help maintain acceptable response times across major workflows.

2.2.19 Web Application Development

As Medrotic is a web-based healthcare platform, the system will be developed using Angular.js **[5]** for the frontend, ASP.NET Core **[6]** for backend APIs and business logic, Python FastAPI services for AI processing, and MS SQL Server for relational data storage. This technology stack supports responsive web delivery, modular service integration, secure backend processing, and structured storage of healthcare data. The selected stack also reflects the project's emphasis on practical, enterprise-style development using accessible and extensible technologies.

2.2.20 Documentation

Prepare comprehensive documentation for all major project phases and deliverables, including the Scope Document, SRS, SDD, Project Report, and supporting technical descriptions. Documentation should also include architectural explanations, design models, use case descriptions, traceability information, and developer-oriented guidance where necessary. Proper documentation is essential to support academic evaluation, future maintenance, and onboarding of new contributors or reviewers.

2.2.21 Testing and Quality Assurance

Conduct thorough testing activities to verify that all major modules of Medrotic operate correctly and meet the stated requirements. Testing should include unit testing, functional testing, integration testing, system testing, manual testing, and automated testing where possible. Special attention should be given to authentication, CNIC-based search, AI service workflows, approval logic, notifications, and record consistency to ensure that the final system is reliable and ready for demonstration and evaluation.

2.2.22 User Training and Support Materials

Provide user-oriented support materials that help Super Admins, Doctors, and Patients understand how to use the system effectively. These materials may include user guides, walkthroughs, FAQs, dashboard instructions, and basic onboarding content. Because Medrotic serves multiple user groups with different technical backgrounds, support materials are necessary to improve usability and reduce confusion during adoption.

2.2.23 Maintenance, Updates and Continuous Improvement

Develop a long-term plan for maintaining and improving Medrotic after the initial implementation. This includes fixing bugs, refining AI workflows, updating security controls, improving performance, and adding new features based on stakeholder feedback. The modular design of the system should support future expansion such as mobile applications, deeper EHR integration, telehealth connectivity,

predictive analytics, and broader healthcare interoperability without requiring complete redesign. The roadmap should also define regular monitoring, user training, and version release cycles to ensure continuous reliability and user adoption. Clear documentation and support channels must be established to enable smooth upgrades, scalability, and long-term sustainability of the platform.

2.3 Current System

In many healthcare institutions, the current system is either manual, partially digitized, or fragmented across departments. Paper files, isolated storage systems, and non-integrated tools create delays when retrieving records or interpreting patient history. Doctors often must depend on manually provided information, which can affect speed and accuracy. Patients may not have direct access to their own complete history, and hospital-level insights may require manual reporting instead of real-time dashboards.

Such systems also struggle with diagnostic documents. Prescriptions may need to be manually interpreted and entered digital form, while X-rays and test reports may follow separate, disconnected workflows. These limitations motivated the design of Medrotic as a centralized digital alternative where records, AI analysis, notifications, and analytics work together within one controlled environment.

Chapter 3
Requirement Analysis

3 Requirement Analysis

Requirement analysis for Medrotic was carried out to identify all actors, workflows, constraints, and system behaviors needed to build a correct, secure, and useful healthcare platform. The approved SRS uses UML use case diagrams and detailed use case descriptions to represent interactions among patients, doctors, and administrators, while also defining the scope of features available to each role in the system. These requirements provided the functional basis for the design models, implementation structure, and testing strategy later documented in the SDD and Project Report and helped ensure that each major module remained directly traceable to a defined user need and system objective.

3.1 Use Case Diagrams

The requirement analysis of Medrotic was carried out through use case modeling to clearly identify system actors and their interactions. The main actors are Super Admin **[Fig 3.3]**, Doctor **[Fig 3.1]**, and Patient **[Fig 3.2]**, and each actor has a distinct set of responsibilities and access rights based on role. These use case diagrams provide a clear visual summary of how each user interacts with the system, define the boundaries of available functionality, and serve as a foundation for both system design and implementation planning [4].

3.1.1 Use Case Diagram for Doctors

The Use Case Diagram for Doctors represents the major clinical responsibilities supported by Medrotic. It shows the doctor as a central actor who can authenticate into the system, access patient records using CNIC, register new patients when needed, upload prescriptions for OCR processing, upload X-rays for deep learning analysis, review AI-generated results, approve or reject critical outputs, and update medical records accordingly. This diagram highlights the doctor's central role in both patient management and supervised AI-assisted diagnosis, ensuring that important medical outputs are not stored without professional review where approval is required.

The doctor's use case view is particularly important because it connects patient record management with diagnostic automation in one continuous workflow. Through this interaction model, Medrotic reduces clerical burden, accelerates access to medical information, and supports AI-based assistance while still preserving the doctor's authority over final clinical interpretation and record updates. The diagram therefore captures one of the project's core design goals: combining intelligent automation with human accountability in practical healthcare workflows.

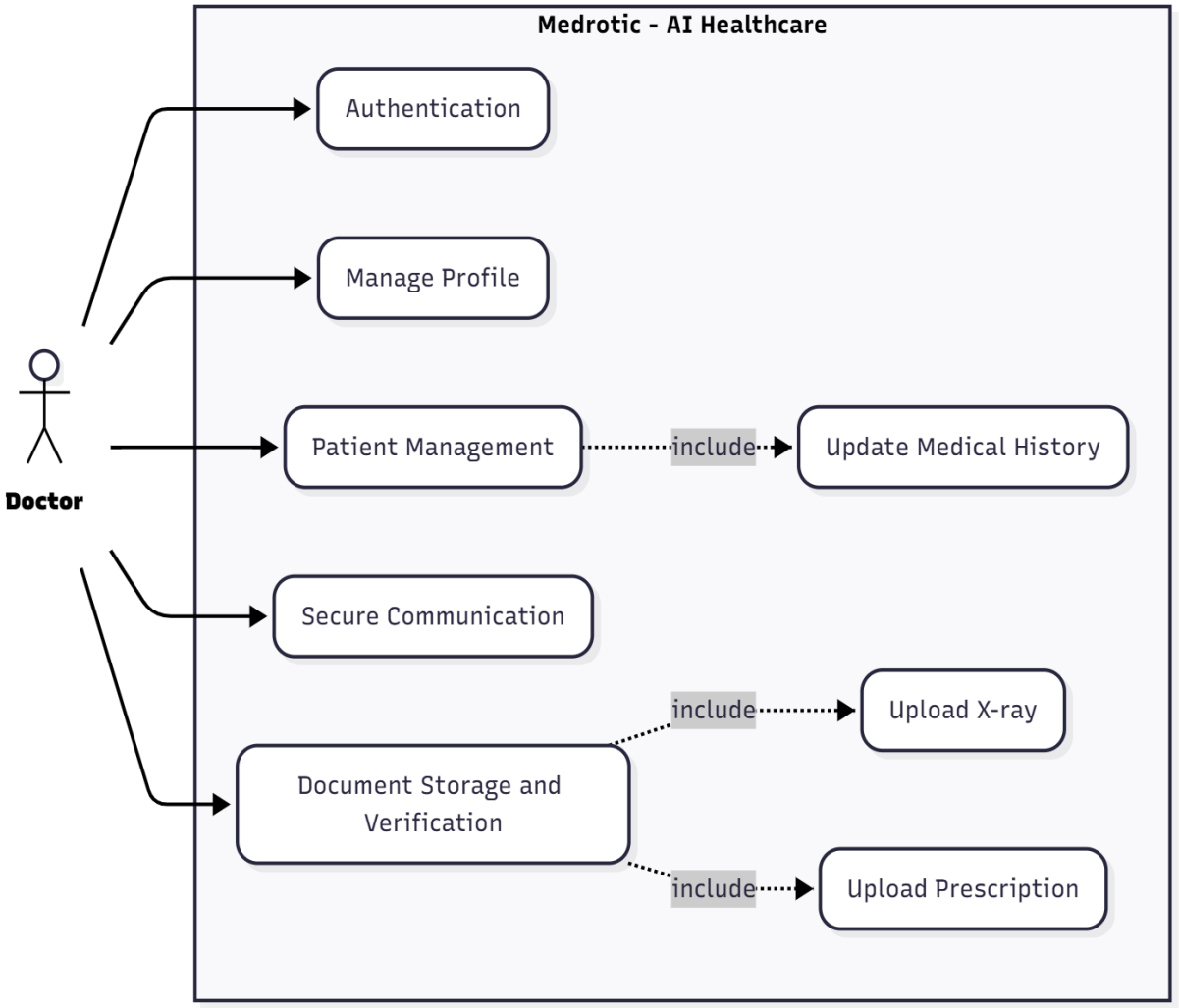


Figure 3. 1: Use Case Diagram for Doctors

3.1.2 Use Case Diagram for Patients

The Use Case Diagram for Patients reflects the self-service orientation of Medrotic from the patient perspective. It includes patient self-registration through CNIC-based identity handling, viewing medical history, adding personal health metrics, interacting with the AI chatbot, searching for doctors, and uploading test reports for automated analysis. This diagram highlights the project's effort to move patients from passive recipients of care to active participants in managing their health information.

By providing patients with direct controlled access to their own records and selected data-entry capabilities, the system improves transparency and engagement. The inclusion of chatbot interaction and doctor search also shows that Medrotic is not limited to record storage; it supports guidance, exploration, and continuity of care. This patient-centric design is one of the major distinguishing features of the project compared with many traditional hospital systems.

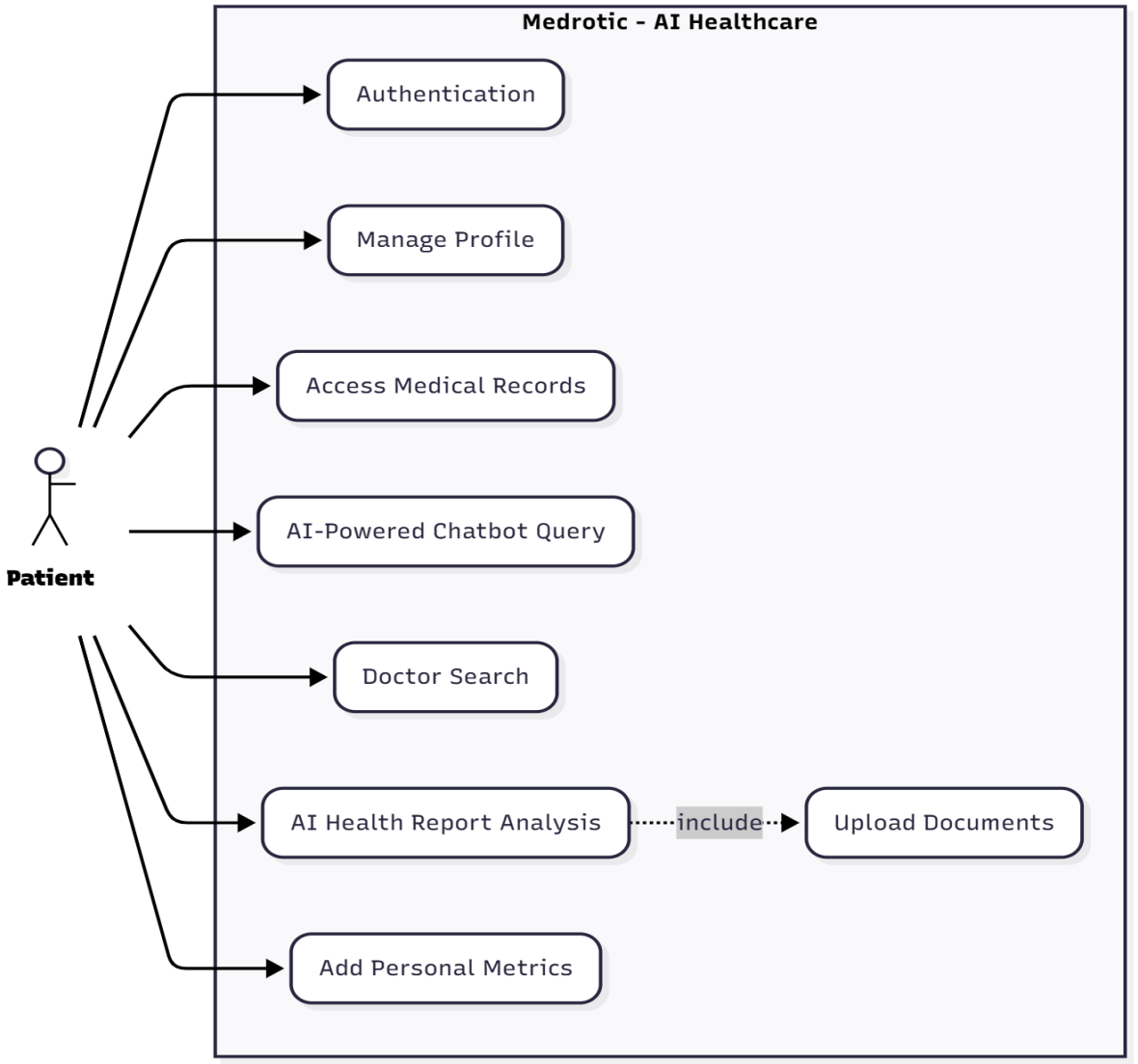


Figure 3. 2: Use Case Diagram for Patients

3.1.3 Use Case Diagram for Admin

The Use Case Diagram for Super Admin (Hospital Administrator) depicts high-level administrative functions essential for institutional governance. The super admin can securely log into the system, register new doctors with complete professional details (PMDC number, specialization), verify doctor credentials before activation, manage hospital profiles and configurations, and monitor real-time analytics including KPIs such as patient counts, doctor activity, document processing volumes, and system health metrics.

This diagram establishes the governance layer necessary for hospital-level oversight, ensuring only authorized medical professionals gain access while providing administrators with comprehensive visibility into platform usage and operational performance. The admin role acts as the control point

for doctor onboarding and system-wide monitoring, creating clear separation between clinical operations and institutional management.

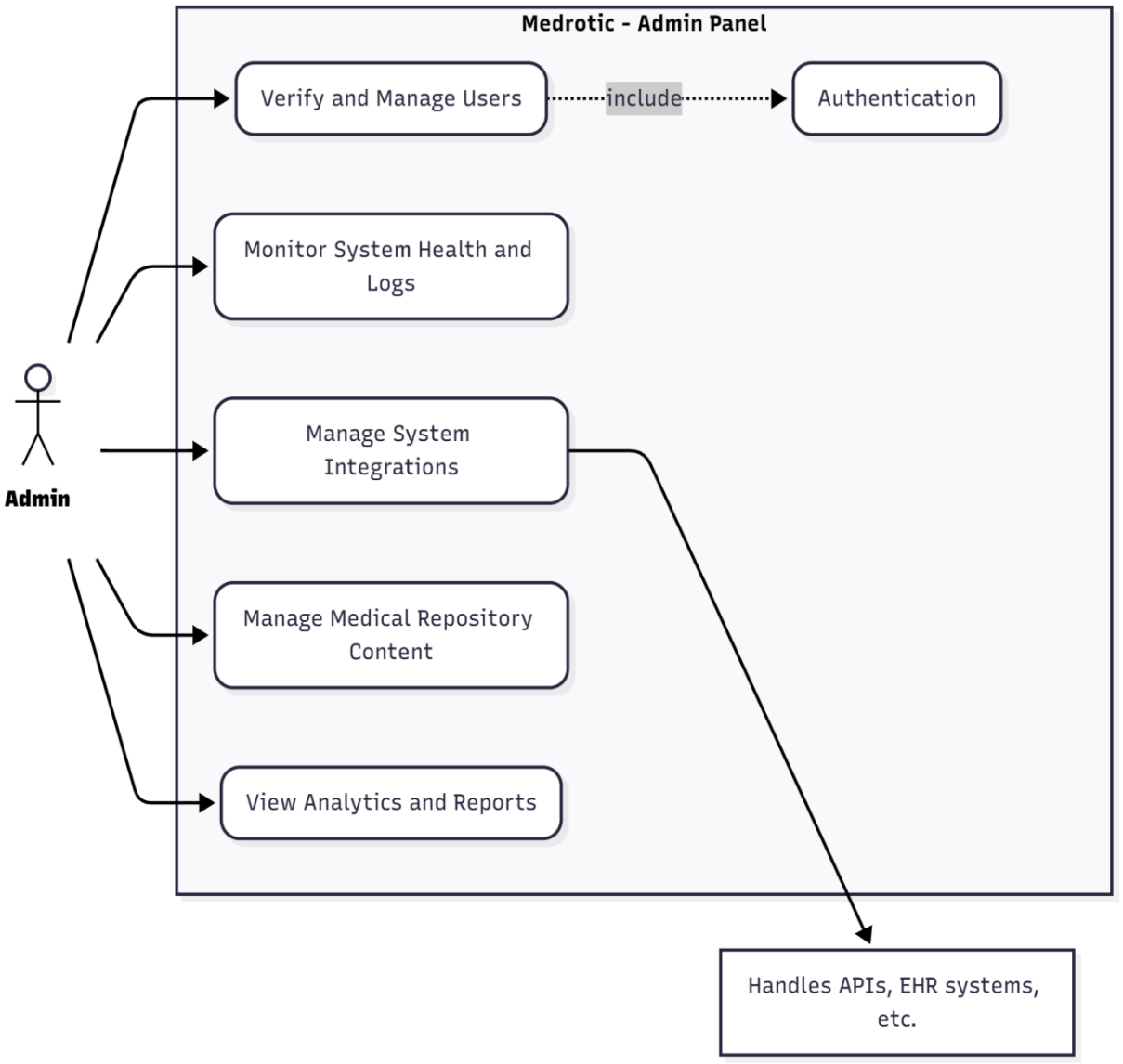


Figure 3. 3: Use Case Diagram for Admin

3.2 Detailed Use Case

The system includes several important use cases that define how different users interact with Medrotic. For example, when a doctor uploads an X-ray, the system sends the image to the AI module for analysis and then asks for doctor approval if the result is critical. Similarly, when a patient uploads a non-critical report, the system processes it through OCR or deep learning and saves the result automatically if no approval is required. Another major use case is patient access via CNIC, where doctors can quickly retrieve patient information using a unique identifier. The Super Admin use cases focus on

hospital oversight, doctor management, and analytics dashboards. These use cases formed the basis of the functional decomposition used throughout the design and implementation stages.

Table 3.2 1: Register Doctor

Use Case ID:	UC-1
Use Case Name:	Register Doctor
Actors:	Primary Actor: Super Admin (Hospital)
Description:	The Super Admin registers a new doctor in the system by providing details such as name, specialty, experience, and contact information.
Trigger:	Super Admin selects the option to add a new doctor.
Preconditions:	PRE-1. Super Admin is logged in. PRE-2. Doctor details are validated for uniqueness.
Postconditions:	POST-1. Doctor records are created in the database. POST-2. Notification is sent to the doctor.
Normal Flow:	1. Super Admin enters doctor details. 2. System validates input. 3. System saves doctor profile. 4. System confirms registration.
Alternative Flows:	1.1 If details are invalid, system prompts correction.
Exceptions:	1.0.E1 Duplicate doctor ID, system alerts and cancels.
Business Rules:	BR-1 Doctors must have valid medical license numbers.
Assumptions:	Assume all doctors are affiliated with the hospital.

Table 3.1.2 Access Patient Data

Use Case ID:	UC-2
Use Case Name:	Access Patient Data
Actors:	Primary Actor: Doctor
Description:	Doctor uses patient's CNIC to access, view, edit, or add medical details.
Trigger:	Doctor enters patient CNIC in search field.
Preconditions:	PRE-1. The doctor is logged in. PRE-2. CNIC is in valid format.
Postconditions:	POST-1. Patient data is displayed or updated.
Normal Flow:	1. Doctor inputs CNIC.

	2. System retrieves patient profile. 3. Doctor edits/adds data. 4. System saves changes.
Alternative Flows:	2.1 If a patient is not found, redirect it to registration.
Exceptions:	1.0.E1 Invalid CNIC, system shows error.
Business Rules	BR-1 Access is limited to authorized doctors.
Assumptions:	CNIC is a unique identifier.

Table 2.2.3 Upload Prescription

Use Case ID:	UC-3
Use Case Name:	Upload Prescription
Actors:	Primary Actor: Doctor
Description:	Doctor uploads a prescription image, and the system applies OCR to extract details and add to patient's database.
Trigger:	Doctor selects upload prescription option.
Preconditions:	PRE-1. Doctor is logged in and has patient profile open. PRE-2. Image is in supported format.
Postconditions:	POST-1. Extracted data is added to patient records automatically.
Normal Flow:	1. Doctor uploads images. 2. System processes with OCR (Mistral models). 3. System adds details to database. 4. Doctor reviews if needed.
Alternative Flows:	None.
Exceptions:	1.0.E1 Image unclear, system requests re-upload.
Business Rules	BR-1 Data added must be accurate; no manual override without doctor approval.
Assumptions:	Assume high-quality images for OCR accuracy.

Table 3.2.3 Upload X-ray

Use Case ID:	UC-4
Use Case Name:	Upload X-ray
Actors:	Primary Actor: Doctor

Description:	Doctor uploads X-ray images, system uses deep learning for disease confirmation, requiring doctor approval to add to patient details**Invalid source specified.**.
Trigger:	The doctor selects an upload X-ray option.
Preconditions:	PRE-1. Doctor is logged in and has patient profile open.
Postconditions:	POST-1. Image stored; analysis output approved and added.
Normal Flow:	1. Doctor uploads images. 2. System analyzes with deep learning. 3. Doctor reviews and approves output. 4. System updates patient record.
Alternative Flows:	3.1 Doctor rejects output, no update.
Exceptions:	1.0.E1 Invalid image format, error message.
Business Rules	BR-1 Doctor approval mandatory for AI outputs.
Assumptions:	Models trained in sufficient data for accuracy.
Business Rules	BR-1 Doctor approval mandatory for AI outputs.
Assumptions:	Models trained in sufficient data for accuracy.

Table 3.2.4 Patient Registration

Use Case ID:	UC-5
Use Case Name:	Patient Registration
Actors:	Primary Actor: Patient
Description:	Patients register themselves using CNIC and personal details.
Trigger:	Patient accesses registration page.
Preconditions:	PRE-1. CNIC is not already registered.
Postconditions:	POST-1. Patient account created.
Normal Flow:	1. Patients enter details including CNIC. 2. System validates. 3. System creates profile.
Alternative Flows:	None.
Exceptions:	1.0.E1 Duplicate CNIC, prompt to login.
Business Rules	BR-1 CNIC must be 13 digits.
Assumptions:	Patients have valid CNIC.

Table 3.2.5 View Medical History

Use Case ID:	UC-6
Use Case Name:	View Medical History
Actors:	Primary Actor: Patient
Description:	Patients view their medical history as added by doctors or self.
Trigger:	Patient logs in and selects historical view.
Preconditions:	PRE-1. Patient is logged in.
Postconditions:	POST-1. History is displayed chronologically.
Normal Flow:	1. Patient requests history. 2. System retrieves and displays.
Alternative Flows:	None.
Exceptions:	None.
Business Rules	BR-1 Only own history accessible.
Assumptions:	Data available in database.

Table 3.2.6 Use AI Chatbot

Use Case ID:	UC-7
Use Case Name:	Use AI Chatbot
Actors:	Primary Actor: Patient
Description:	Patient queries about the AI chatbot linked to their database for health-related questions.
Trigger:	Patient opens chatbot interface.
Preconditions:	PRE-1. Patient is logged in.
Postconditions:	POST-1. Response provided based on database.
Normal Flow:	1. Patient input query. 2. System processes with AI. 3. System responds.
Alternative Flows:	None.
Exceptions:	1.0.E1 Query unclear, ask for clarification.
Business Rules	BR-1 Responses personalized but not medical advice.
Assumptions:	AI models are integrated properly.

Table 3.2.7 Search Doctors

Use Case ID:	UC-8
Use Case Name:	Search Doctors
Actors:	Primary Actor: Patient
Description:	Patient searches for doctors by city, disease, specialty, experience, etc.
Trigger:	Patient selects search option.
Preconditions:	PRE-1. Patient is logged in.
Postconditions:	POST-1. List of matching doctors displayed.
Normal Flow:	1. Patient enters criteria. 2. System searches database. 3. System shows results.
Alternative Flows:	None.
Exceptions:	1.0.E1 No matches, suggest alternatives.
Business Rules	BR-1 Search filters must be accurate.
Assumptions:	Doctors' data up to date.

Table 3.2.8 View Analytics

Use Case ID:	UC-9
Use Case Name:	View Analytics
Actors:	Primary Actor: Super Admin (Hospital)
Description:	Super Admin views real-time metrics like total patients, doctors, and performance KPIs.
Trigger:	Super Admin accesses analytics dashboard.
Preconditions:	PRE-1. Super Admin is logged in.
Postconditions:	POST-1. Metrics and reports displayed with visualizations (charts/graphs).
Normal Flow:	1. Super Admin selects analytics view. 2. System aggregates data. 3. System displays reports.
Alternative Flows:	None.
Exceptions:	1.0.E1 No data available, show empty state message.
Business Rules	BR-1 Data aggregated anonymously for privacy.
Assumptions:	Database has sufficient historical data.

Table 3.2.9 Send Notifications

Use Case ID:	UC-10
Use Case Name:	Send Notifications
Actors:	Primary Actor: System (on behalf of Doctor/Super Admin)
Description:	System sends alerts for updates, approvals, test results, or queries via push, email, or SMS.
Trigger:	Events such as prescription approval or test result availability.
Preconditions:	PRE-1. User has opted for notifications. PRE-2. Contact details are valid.
Postconditions:	POST-1. Notification delivered and logged.
Normal Flow:	1. Event triggers notification. 2. System formats message. 3. System sends via chosen channel.
Alternative Flows:	2.1 If email fails, fall back to SMS.
Exceptions:	1.0.E1 Invalid contact, log error.
Business Rules	BR-1 Notifications compliant with privacy laws (e.g., no sensitive data in SMS).
Assumptions:	Integration with external notification services (e.g., SMTP for email).

Table 3.2.10 Add Personal Metrics

Use Case ID:	UC-11
Use Case Name:	Add Personal Metrics
Actors:	Primary Actor: Patient
Description:	Patients add self-monitored metrics like blood pressure or sugar levels to their history.
Trigger:	Patient selects add metric option in dashboard.
Preconditions:	PRE-1. Patient is logged in.
Postconditions:	POST-1. Metric added to history with timestamp.
Normal Flow:	1. Patients enter metric details. 2. System validates input. 3. System saves database.
Alternative Flows:	None.

Exceptions:	1.0.E1 Invalid format (e.g., non-numeric value), prompt correction.
Business Rules	BR-1 Metrics are patient-editable but doctor-viewable.
Assumptions:	Metrics do not require immediate doctor approval.

Table 3.2.11 Test Report Analysis

Use Case ID:	UC-12
Use Case Name:	Test Report Analysis
Actors:	Primary Actor: Doctor/Patient
Description:	User uploads test report image: system analyzes using deep learning for insights, with doctor approval if uploaded by doctor **Invalid source specified.**.
Trigger:	User selects upload test report option.
Preconditions:	PRE-1. User is logged in. PRE-2. Image in supported format.
Postconditions:	POST-1. Analysis output added to patient records (approved if by doctor).
Normal Flow:	1. User uploads images. 2. System analyzes with deep learning. 3. Output reviewed/approved. 4. System updates record.
Alternative Flows:	3.1 Patient upload: No approval needed; flagged for doctor review.
Exceptions:	1.0.E1 Poor image quality, request re-upload.
Business Rules	BR-1 Critical insights require doctor confirmation.
Assumptions:	Models fine-tuned on datasets from Kaggle/Hugging Face.

3.3 Functional Requirements

The functional requirements of Medrotic define the core capabilities the system must deliver. Key requirements include Super Admin's ability to register and verify doctors, doctors' capacity to access and update patient records via CNIC **[Table 3.2.2]**, automated OCR processing of prescriptions, deep learning analysis of X-rays and test reports with mandatory doctor approval for critical results, patient self-registration **[Table 3.2.3]** and history viewing, AI chatbot support, doctor search functionality, and real-time hospital analytics. All twelve functional requirements identified in the SRS were successfully implemented and mapped to design artifacts **[5]**.

Table 3.3. 1: Doctor Registration

Identifier	FR-1
Title	Doctor Registration
Requirement	The Super Admin shall be able to register a new doctor by entering details including name, specialty, city, experience, and contact info.
Source	Hospital requirements
Rationale	To ensure only authorized doctors are added to the system.
Business Rule	BR-1: Doctor must be verified by hospital.
Dependencies	None.
Priority	High

The system shall allow the Super Admin to register a new doctor by entering the required information such as name, specialty, city, experience, and contact details. This requirement is necessary because doctor participation in the platform must be controlled by the hospital administration rather than left open to uncontrolled enrollment. The requirement also supports the overall governance objective of ensuring that only authorized professionals become part of the Medrotic ecosystem.

From a project perspective, this requirement directly supports the administrative dashboard and is linked to the broader goal of secure role management. The associated business rule requires that the doctor must be verified by the hospital, which makes this feature more than a simple registration form. It functions as the controlled entry point for clinical users and provides the foundation for later workflows such as patient access, AI-assisted diagnosis, and dashboard usage.

Table 3.3. 2: Patient Access via CNIC

Identifier	FR-2
Title	Patient Access via CNIC
Requirement	Upon entering a valid CNIC, the system shall retrieve and display the patient's medical details for the logged-in doctor.
Source	User requirements
Rationale	For quick and secure patient data access.
Business Rule	BR-1: CNIC must be 13 digits.
Dependencies	FR-1
Priority	High

The system should retrieve and display patient medical details when a logged-in doctor enters a valid CNIC. This requirement is essential because Medrotic uses CNIC as the primary patient identity

mechanism, enabling quick, unique, and secure access to medical records. In a healthcare environment where time and accuracy matter, this feature plays a major role in improving operational efficiency and continuity of care.

The requirement also reflects one of the most practical strengths of the proposed platform. Instead of relying on fragmented manual lookup processes, doctors can locate relevant patient information with a single standardized identity field. The business rule that CNIC must be 13 digits ensures validation consistency, while the dependency on doctor access control preserves confidentiality and correct role-based use.

Table 3.3. 3: Patient Registration

Identifier	FR-3
Title	Patient Registration
Requirement	The system shall allow patients or doctors to register a patient using CNIC as the primary key, ensuring uniqueness.
Source	User requirements
Rationale	To enable new patients to join the system securely.
Business Rule	BR-1: CNIC validation required.
Dependencies	None.
Priority	High

The system should allow patients or doctors to register a patient using CNIC as the primary unique key. This supports both self-service onboarding and institution-assisted registration while ensuring that patient identity remains consistent across all workflows. By allowing both patient-driven and doctor-assisted registration, the platform becomes more flexible and practical for diverse healthcare situations.

This requirement also strengthens the project's commitment to patient empowerment. A patient can become part of the system independently, yet the same identity structure still supports continuity when doctors later interact with the record. Because healthcare systems depend heavily on avoiding duplicate records, the uniqueness of the condition attached to CNIC is especially important in this feature.

Table 3.3. 4: Prescription Upload and OCR

Identifier	FR-4
Title	Prescription Upload and OCR

Requirement	The doctor shall upload a prescription image, and the system shall apply OCR using Mistral models to automatically extract and add details to the patient's database.
Source	AI integration requirements
Rationale	To automate data entry from prescriptions.
Business Rule	BR-1: Automated addition without manual input.
Dependencies	FR-2
Priority	Medium

The system shall allow the doctor to upload a prescription image and automatically apply OCR using Mistral-based models to extract text and add the information to the patient database. This requirement reduces the burden of manually re-entering prescription content and supports faster digitization of handwritten or image-based records. It also strengthens data completeness by turning static documents into structured digital information.

In the broader project context, this requirement shows how AI is used in Medrotic not just for advanced diagnostics but also for practical administrative improvement. OCR creates direct value by accelerating record creation and reducing human transcription effort. The associated business rule states that data should be added automatically without manual input, which highlights the automation objective of the feature.

Table 3.3. 5: X-ray Upload and Analysis

Identifier	FR-5
Title	X-ray Upload and Analysis
Requirement	The doctor should upload an X-ray image, and the system should use deep learning models to analyze diseases**Invalid source specified.**, storing the image and requiring doctor approval for output to reflect in patient details.
Source	AI integration requirements
Rationale	To assist in diagnostics with AI confirmation.
Business Rule	BR-1: Doctor approval mandatory.
Dependencies	FR-2
Priority	Medium

The system shall allow the doctor to upload an X-ray image and use deep learning models to analyze it for disease-related findings. The result is generated by the AI service but must be approved by the doctor before it becomes part of the patient’s clinical record. This requirement embodies one of the

core ideas of Medrotic: use AI to support medical work while preserving professional oversight where the stakes are high.

This feature is important because image interpretation is often time-consuming and requires careful review. By assisting doctors with preliminary automated analysis, the system improves efficiency and supports faster clinical workflows. At the same time, the business rule that doctor approval is mandatory ensures that the platform does not position AI as a replacement for clinical authority.

Table 3.3. 6: View Medical History

Identifier	FR-6
Title	View Medical History
Requirement	The patient shall be able to view their medical history added by doctors or themselves in chronological order.
Source	Patient empowerment requirements
Rationale	To provide patients with access to their health records.
Business Rule	None.
Dependencies	FR-3
Priority	High

The system allows the patient to view medical history entries added by doctors or by the patient in chronological order. This requirement is essential for transparency, continuity of care, and patient empowerment. It transforms patients from passive recipients of care into informed users who can review their own medical timeline through a single platform.

A chronological presentation of records is especially important because medical understanding often depends on sequence. Medrotic therefore does more than simply store records; it organizes them in a meaningful way that supports both recall and future clinical use. This feature also complements patient-uploaded metrics and reports by showing how multiple data sources contribute to a unified health history.

Table 3.3. 7: Add Personal Medical History

Identifier	FR-7
Title	Add Personal Medical History
Requirement	The patient should add personal metrics like sugar test or blood pressure to their history.
Source	Patient empowerment requirements
Rationale	For self-monitoring and complete records.
Business Rule	None.

Dependencies	FR-3
Priority	Medium

The system shall allow the patient to add personal metrics such as sugar readings or blood pressure into their history. This requirement supports self-monitoring and creates a more complete medical record that includes patient-generated health information alongside clinician-generated entries. It is particularly useful for conditions where routine monitoring outside the hospital is important.

The feature also aligns with the patient-centered design of the project. Rather than limiting the patient to viewing data only, Medrotic enables meaningful participation in record enrichment. This adds long-term value because personal metrics can provide context for future diagnoses and clinical decisions.

Table 3.3. 8: AI Chatbot for Queries

Identifier	FR-8
Title	AI Chatbot for Queries
Requirement	The system shall provide an AI chatbot linked to the patient's database for answering health queries.
Source	AI integration requirements
Rationale	To offer 24/7 support for patient questions.
Business Rule	BR-1: Responses based on database data.
Dependencies	FR-3
Priority	Medium

The system should provide an AI chatbot linked to the patient’s database for answering health-related queries. This feature adds an intelligent conversational layer to the patient dashboard and extends system accessibility beyond static forms and record pages. It can help users navigate information, ask simple questions, and obtain contextual support at any time.

This requirement is important because modern healthcare systems increasingly need responsive interfaces that improve patient engagement. The business rule states that responses should be based on database data, which means the chatbot is expected to remain context-aware and relevant rather than purely generic. In Medrotic, the chatbot supports the broader goal of making healthcare information more available and interactive.

Table 3.3. 9: Test Report Analysis

Identifier	FR-9
Title	Test Report Analysis

Requirement	The system shall analyze uploaded test reports using deep learning for insights.
Source	AI integration requirements
Rationale	To automate test result interpretation.
Business Rule	BR-1: Doctor oversight if critical.
Dependencies	FR-2
Priority	Medium

The system shall analyze uploaded test reports using AI for insights. This requirement extends Medrotic's intelligence beyond prescriptions and X-rays by enabling support for other report-driven diagnostic workflows. Depending on the scenario, doctor oversight may apply for critical outputs, ensuring controlled clinical handling.

The practical significance of this feature lies in improving the speed with which report contents can become part of patient understanding and clinical context. It also supports the project's broader idea that different types of medical documents should pass through streamlined digital workflows instead of remaining isolated files.

Table 3.3. 10: Doctor Search

Identifier	FR-10
Title	Doctor Search
Requirement	The patient shall search for doctors by criteria such as city, disease, specialty, experience.
Source	Patient empowerment requirements
Rationale	To help patients find suitable doctors.
Business Rule	None.
Dependencies	FR-1
Priority	Medium

The system shall allow patients to search for doctors using criteria such as city, disease, specialty, and experience. This requirement strengthens patient convenience and decision support by allowing users to discover appropriate healthcare providers through the same platform in which they manage records and reports.

In terms of user experience, this feature makes Medrotic more than an internal hospital tool. It becomes a patient-facing system that supports health service navigation as well. This is valuable because it

integrates patient information management with access to care pathways rather than keeping them as separate digital experiences.

Table 3.3. 11: View Hospital Analytics

Identifier	FR-11
Title	View Hospital Analytics
Requirement	Super Admin shall view real-time KPIs like patient/doctor counts and performance metrics with visualizations.
Source	Hospital oversight requirements
Rationale	For data-driven decision-making and resource management.
Business Rule	BR-1: Anonymized data for compliance.
Dependencies	FR-1, FR-3
Priority	High

The system shall allow the Super Admin to view real-time KPIs such as patient counts, doctor counts, and performance metrics through visualizations. This requirement is essential for administrative oversight because hospitals need operational insight to make informed resource and policy decisions. In Medrotic, analytics is not merely decorative. It provides value by summarizing institutional activity in a form that can support monitoring and planning. The SRS notes anonymized data for compliance as a business rule, which indicates that analytics must remain useful without compromising privacy obligations.

Table 3.3. 12: Notification Delivery

Identifier	FR-12
Title	Notification Delivery
Requirement	The system shall send real-time notifications (push/email/SMS) for events like approvals or updates.
Source	Communication requirements
Rationale	To keep users informed and improve response times.
Business Rule	BR-1: Opt-in required; compliant with privacy standards.
Dependencies	FR-2, FR-3
Priority	Medium

The system shall send real-time notifications through push, email, or SMS for events such as approvals and updates. This requirement ensures that the platform remains reactive and communicative, especially when important clinical or administrative actions occur.

Notifications are important because healthcare workflows often depend on timely awareness rather than passive checking. Whether a doctor needs to review an AI result or a patient needs to know that an update has occurred, communication must happen quickly and through appropriate channels. The SRS also notes privacy-compliant opt-in behavior, which keeps the feature aligned with responsible user communication.

3.4 Non-Functional Requirements

The non-functional requirements ensure that Medrotic is usable, secure, scalable, reliable, and maintainable. The system must provide a simple and intuitive interface for all users so that registration, search, and record access can be completed quickly without excessive training. The web application must also perform efficiently with fast page loads and responsive AI processing.

Security is a major non-functional concern because the platform handles sensitive medical information **[Table 3.4.3]**. Therefore, the system must use encrypted communication, role-based access control, secure authentication, and logging for accountability. The system should also be scalable enough to handle multiple users and a large amount of medical data while remaining stable and maintainable over time.

3.4.1 Usability

The system must be easy to learn and efficient to use for patients, doctors, and administrators. According to the SRS, users should be able to retrieve a previous medical history item or last uploaded document with a simple interaction, and new users should be able to complete registration and first login within two minutes without prior training. These requirements reflect the need for intuitive interface design and low learning effort.

In Medrotic, usability is especially important because the user base is diverse. Patients may have limited technical experience, while doctors need fast access without unnecessary interface complexity. For this reason, the system uses role-specific dashboards, clear navigation, and form-driven workflows tailored to the needs of each actor.

Table 3.4. 1 Usability

Identifier	NFR-1
Title	Usability
Requirement	Users should retrieve medical history and complete registration/login through intuitive workflows with minimal training.

Importance	High usability is essential for adoption across different user groups.
Project Interpretation	Medrotic uses simple forms, role-specific dashboards, and structured navigation.

3.4.2 Performance

The system must perform efficiently under normal and moderately high load. The SRS specifies that 95 percent of webpages should download completely within four seconds on a 20 Mbps or faster connection, AI processing for images under 5 MB should complete within ten seconds, and the platform should support up to 1,000 concurrent users while maintaining database query response times under two seconds. Analytics should refresh every 30 seconds without user intervention.

These performance expectations shaped several design choices in Medrotic, including modular service separation, efficient database design, and support for analytics caching in the architectural and traceability materials. Performance is particularly important because delays in healthcare workflows can directly affect usability and trust in the system.

Table 3.4. 2 Performance

Identifier	NFR-2
Title	Performance
Requirement	Fast webpage loading, timely AI analysis, support for concurrency, and periodic analytics refresh.
Importance	Essential for responsive healthcare interaction and user confidence.
Project Interpretation	Multi-tier architecture and optimized queries support responsiveness.

3.4.3 Security

The system must protect sensitive healthcare data through secure communication and controlled access. The SRS requires HTTPS encryption for data in transit, RBAC to restrict actions according to user role, secure storage of sensitive data such as CNIC, audit logging, and additional controls such as MFA for critical operations. These expectations align with the general security posture required for healthcare systems.

In Medrotic, security is integrated into the platform's structure rather than added as an afterthought. Authentication, role management, audit-friendly workflow design, and controlled approvals all reflect

this requirement. Since patient privacy and data integrity are central to system trust, security remains one of the most important non-functional dimensions of the project.

Table 3.4. 3 Security

Identifier	NFR-3
Title	Security
Requirement	The system should protect data through encryption, RBAC, secure storage, and auditing.
Importance	Critical due to sensitive medical information.
Project Interpretation	HTTPS, role separation, and secure backend design support this requirement.

3.4.4 Scalability

The system must support growth from a smaller user base to much larger deployment conditions without unacceptable degradation. The SRS states that Medrotic should scale horizontally using Docker containers to accommodate growth from 100 to 10,000 users, while the database should support partitioning for large data volumes. This requirement is important because healthcare adoption can expand gradually across providers and institutions.

In project terms, scalability influenced the modular and service-oriented structure of the application. Separating AI processing, application logic, and data management into distinct layers makes it easier to scale specific components when load increases. This design ensures that Medrotic can evolve from a limited deployment into a broader operational platform if future enhancements are added.

Table 3.4. 4 Scalability

Identifier	NFR-4
Title	Scalability
Requirement	The platform shall support horizontal growth and large-scale data handling.
Importance	Necessary for future institutional expansion.
Project Interpretation	Docker-oriented modularity and layered architecture support scaling.

3.4.5 Reliability

The system must remain dependable during routine healthcare operations. The SRS specifies 99 percent uptime with automated backups every 24 hours and fallback manual modes if AI analysis fails. These requirements recognize that healthcare software cannot depend entirely on ideal conditions and must remain useful even when one service or input path becomes problematic.

Reliability in Medrotic is reflected in approval workflows, audit-capable records, backup awareness, and fallback expectations. If an image is poor or AI confidence is insufficient, the system still preserves human review pathways. This improves resilience and ensures that automation never becomes a single point of failure in care-related processes.

Table 3.4. 5 Reliability

Identifier	NFR-5
Title	Reliability
Requirement	The system shall maintain high uptime, backups, and fallback modes for failed AI analysis.
Importance	Required for continuity in healthcare use.
Project Interpretation	Workflow control and resilient system behavior support reliability.

3.4.6 Maintainability

The system must be easy to update, test, and extend overtime. The SRS requires modular design through microservices architecture and emphasizes automated testing with frameworks such as xUnit and PyTest to reduce regression risk during future changes. Maintainability is especially important for a project like Medrotic because both healthcare workflows and AI components may evolve after initial deployment.

The project's architecture and documentation structure directly support this requirement. Requirements were separated clearly in the SRS, design decisions were documented in the SDD, and implementation was divided into frontend, backend, AI, and data modules. This creates a strong foundation for future upgrades such as mobile support, telehealth integration, and broader clinical features.

Table 3.4. 6 Maintainability

Identifier	NFR-6
Title	Maintainability
Requirement	The codebase shall remain modular, documented, and testable for future updates.
Importance	Important for long-term enhancement and academic software quality.
Project Interpretation	Multi-tier modular design and documented components improve maintainability.

3.4.7 Browser Compatibility

The platform must remain accessible through modern web browsers including Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. Since Medrotic is primarily a web application, browser compatibility is necessary to ensure availability across diverse user environments. This requirement also supports the project's broader accessibility and deployment goals.

The use of Angular.js and standard web technologies helps satisfy this requirement by targeting browser-delivered responsive interfaces. In practice, this means users do not require specialized software installation to interact with the platform, which makes adoption easier for patients, hospitals, and doctors.

Table 3.4. 7 Browser Compatibility

Identifier	NFR-7
Title	Browser Compatibility
Requirement	The system should operate correctly on modern browsers.
Importance	Necessary for broad user accessibility.
Project Interpretation	Web-based responsive frontend supports this requirement.

3.4.8 Communication Interfaces

The platform must support secure communication through APIs and channels such as SMTP, SMS, HTTPS, and WebSockets. The SRS explains that secure API authentication, encrypted communication, and real-time updates are part of the system's communication expectations. These

interfaces support chatbot responsiveness, notification delivery, analytics refresh, and interaction between layers and external services.

This requirement is relevant because Medrotic depends on multiple communication paths rather than a single local workflow. AI services need structured API exchange, the frontend must consume backend APIs, and notifications must be delivered through suitable user channels. The result is a system that is both interactive and service-integrated, which is essential for a modern healthcare platform.

Table 3.4. 8 Communication Interfaces

Identifier	NFR-8
Title	Communication Interfaces
Requirement	The system shall support secure APIs, real-time updates, email, and SMS communication.
Importance	Supports integrated system behavior and timely alerts.
Project Interpretation	REST APIs, WebSockets, SMTP, and notification workflows support this requirement.

Chapter 4
Design and Architecture

4 Design and Architecture

The design and architecture of Medrotic were developed to satisfy the functional and non-functional requirements identified during requirement analysis while keeping the system modular, secure, scalable, and easy to extend. Since the platform is intended for real healthcare use, the architecture needed to support both routine information management and AI-assisted medical processing without making the system difficult to maintain or modify later. For that reason, the approved SDD presents Medrotic as a multi-tier application in which presentation, business logic, AI services, data storage, and external communication are separated into distinct layers.

This separation of concerns is important because healthcare software typically deals with sensitive data, multiple user roles, and workflows that may change over time. By isolating major responsibilities into different tiers, Medrotic can support independent development, testing, and scaling of each module without affecting the entire system. This also improves readability of the system design, reduces coupling between components, and makes future enhancement such as new AI models, additional dashboards, or extra notification channels much easier to introduce.

The architecture also reflects the nature of the project itself. Medrotic is not just a record-keeping system; it combines user management, medical document handling, analytics, and intelligent analysis in one platform. Therefore, the design had to ensure that doctors, patients, and administrators could each interact with the system through role-specific views while the backend handled secure data processing, validation, and workflow control in a consistent manner.

4.1 System Architecture

Medrotic follows a clean multi-tier client-server architecture. The client tier consists of Angular.js dashboards designed for the three main roles: Super Admin, Doctor, and Patient. These dashboards are responsible for user interaction, validation at the interface level, and presentation of analytics, forms, histories, and AI-driven outputs. The application tier, developed with ASP.NET Core, manages business logic, authentication, authorization, workflow decisions, and RESTful APIs **[6]**.

The AI processing tier, implemented through Python FastAPI services, performs OCR and deep learning tasks for uploaded prescriptions, X-rays, and test reports. The data tier uses MS SQL Server to store users, records, documents, AI outputs, and notifications. The external services tier supports communication and integration through channels such as email, SMS, and real-time update services.

This architecture ensures that Medrotic remains maintainable, secure, and expandable as more features are introduced.

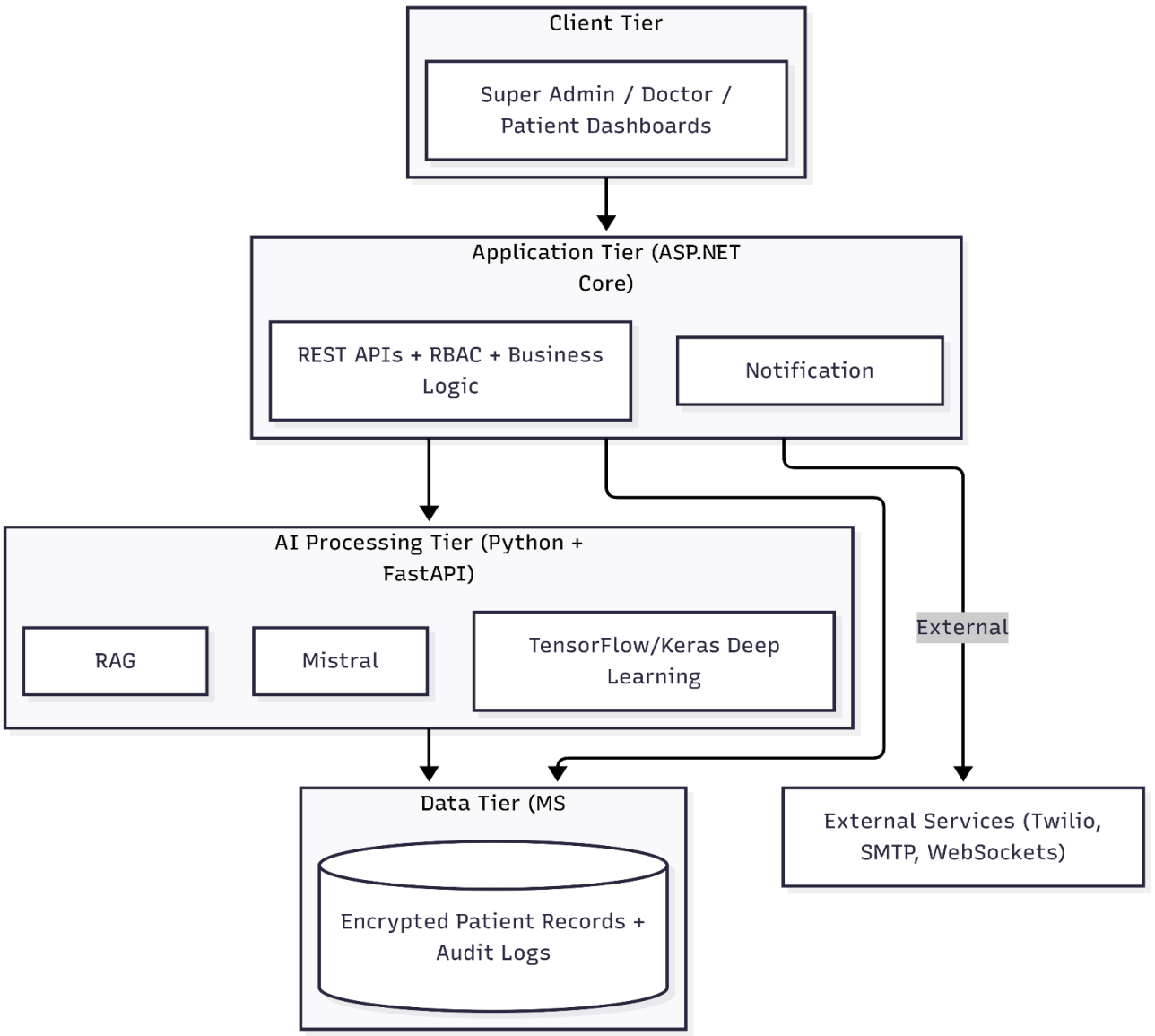


Figure 4. 1: System Architecture for Medrotic

4.2 Data Representation

In Medrotic, data is stored using a structured, secure, and object-oriented database design that supports the efficient management of patients, doctors, hospital administrators, medical records, uploaded documents, AI-generated results, and notifications. The system transforms its object-oriented model into a relational database through Entity Framework Core using a code-first approach, while inheritance among users is implemented through Table-Per-Hierarchy mapping with a role discriminator. This structured representation ensures that all major modules of the system, including

Super Admin, Doctor, Patient, AI Processing, and Notification handling, can store, retrieve, and update information accurately and consistently across the platform.

The central data entity in Medrotic is the User structure, which stores common information such as CNIC, full name, email, password hash, phone number, role, and account creation timestamp. From this common structure, specialized roles such as Patient, Doctor, and Super Admin are derived, allowing the system to maintain both shared identity information and role-specific data in an organized way. This design supports authentication, authorization, and role-based access control while also reducing redundancy in the storage of user-related information.

The Patient and Doctor modules use dedicated tables to hold role-specific details required by the healthcare environment. The Patient entity stores information such as date of birth, gender, blood group, and emergency contact, which allows the system to support personalized health management and medical record association. The Doctor entity stores specialization, PMDC number, and verification status, which helps the platform enforce hospital-approved participation and makes doctor search and administrative verification possible. The Super Admin entity, although simpler, exists as a separate specialized role to support governance, registration, and system-wide oversight.

A major part of the Medrotic data model is the Medical Records structure, which maintains chronological healthcare history for each patient. Each medical record is linked to a patient and stores information such as diagnosis, treatment, creation time, and the role of the creator, whether doctor or patient. This representation allows the system to build a continuous patient history over time and supports features such as viewing medical history, adding personal metrics, and appending results generated from AI-assisted workflows.

The Documents and AI Results modules are especially important in Medrotic because they support the platform's intelligent healthcare features. The Documents table stores uploaded files such as prescriptions, X-rays, and lab reports along with metadata including patient ID, document type, uploader role, upload time, file path, and current workflow state. The AI Results table stores OCR-extracted text, confidence scores, generated findings, approval status, and doctor review information. This structure allows Medrotic to manage document lifecycle states such as uploaded, analyzed, pending approval, approved, or rejected while preserving full traceability between uploaded files and AI-generated outputs.

The Notification module stores alert-related data so that patients, doctors, and administrators can receive timely updates on important events in the system. Each notification record may be linked to a

medical record and contains message content, delivery channel such as push, email, or SMS, sent timestamp, and read status. This structured storage allows the system to support both immediate communication and later reference, ensuring that important updates remain visible and auditable.

The structured nature of the Medrotic database also enables efficient system processing. When a doctor uploads a prescription, X-ray, or test report, the system creates a document entry, routes it to the appropriate AI workflow, stores the resulting OCR or deep learning output, and then updates the approval and medical history records accordingly. Similarly, when a patient adds personal metrics or uploads a report, the related tables are updated in a controlled way, and relevant notifications may be triggered automatically. This makes data movement across modules smooth, traceable, and aligned with the project's role-based workflow design.

To ensure performance and integrity, Medrotic uses indexing on important searchable attributes such as CNIC, PatientId, DocumentType, and CurrentState. CNIC serves as the primary search key across the system and is heavily indexed to allow fast and secure patient retrieval, especially for doctor-facing workflows. Foreign key constraints, normalized table structures, and clearly defined relationships further reduce duplication, maintain consistency, and improve query performance when handling large volumes of healthcare data.

The data representation also supports security and compliance requirements. Sensitive data is handled through secure storage mechanisms, encrypted communication channels, and role-restricted access patterns. Audit-friendly structures such as approval fields, timestamps, creator roles, and document states help preserve accountability and ensure that healthcare operations remain transparent and controlled. This is especially important in Medrotic because AI-generated outputs must often remain linked to doctor approval before becoming part of the official patient record.

Overall, the data representation in Medrotic does not merely serve as a storage mechanism; it acts as the backbone of the entire system. It enables seamless coordination among users, records, AI modules, analytics, and notifications while ensuring consistency, security, scalability, and efficient retrieval of medical information. Through its structured and role-aware design, the database supports the smooth functioning of Medrotic as a modern AI-powered healthcare management platform.

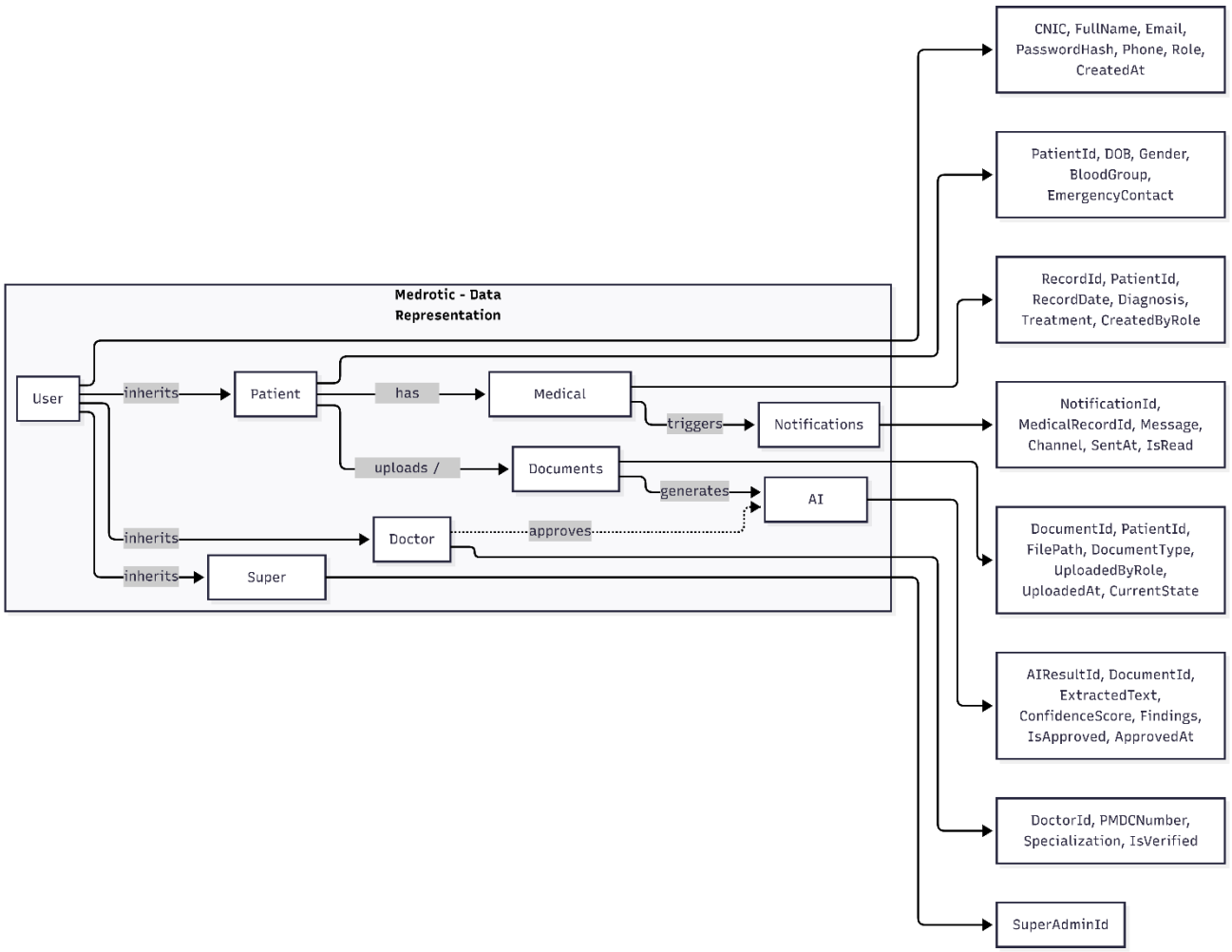


Figure 4. 2: Data Representation Diagram of Medrotic

4.3 Process Flow

The process flow representation of Medrotic illustrates how the main users and system components interact in a structured sequence. It provides a clear view of how doctors, patients, super admins, and the AI-backed system work together to manage healthcare records, diagnostic documents, and administrative tasks in a controlled environment.

In the Doctor workflow, the process begins when a doctor logs into the system and searches for a patient using CNIC. After accessing the patient's profile, the doctor can upload a prescription, X-ray, or test report. The system then routes the uploaded document through the appropriate AI service, where OCR or deep learning analysis is performed. For critical outputs such as X-rays, the result remains pending until the doctor reviews and approves it. Once approved, the system updates the patient's medical record and sends notification if required.

In the Patient workflow, the patient may register, view medical history, upload reports, add personal health metrics, and search for doctors. Uploaded non-critical reports are analyzed automatically, while

patient-entered data is stored chronologically in the medical record. The system also supports chatbot interaction and status updates, enabling patients to remain informed about their records and activities without needing repeated manual follow-up.

In the Super Admin workflow, the administrator manages doctor accounts and monitors hospital-level analytics. The admin can register and verify doctors, view system activity, and oversee overall platform usage. These functions ensure that the system remains governed, traceable, and suitable for institutional use. Overall, the process flow shows that Medrotic is designed to automate routine work while preserving human control in sensitive healthcare actions.

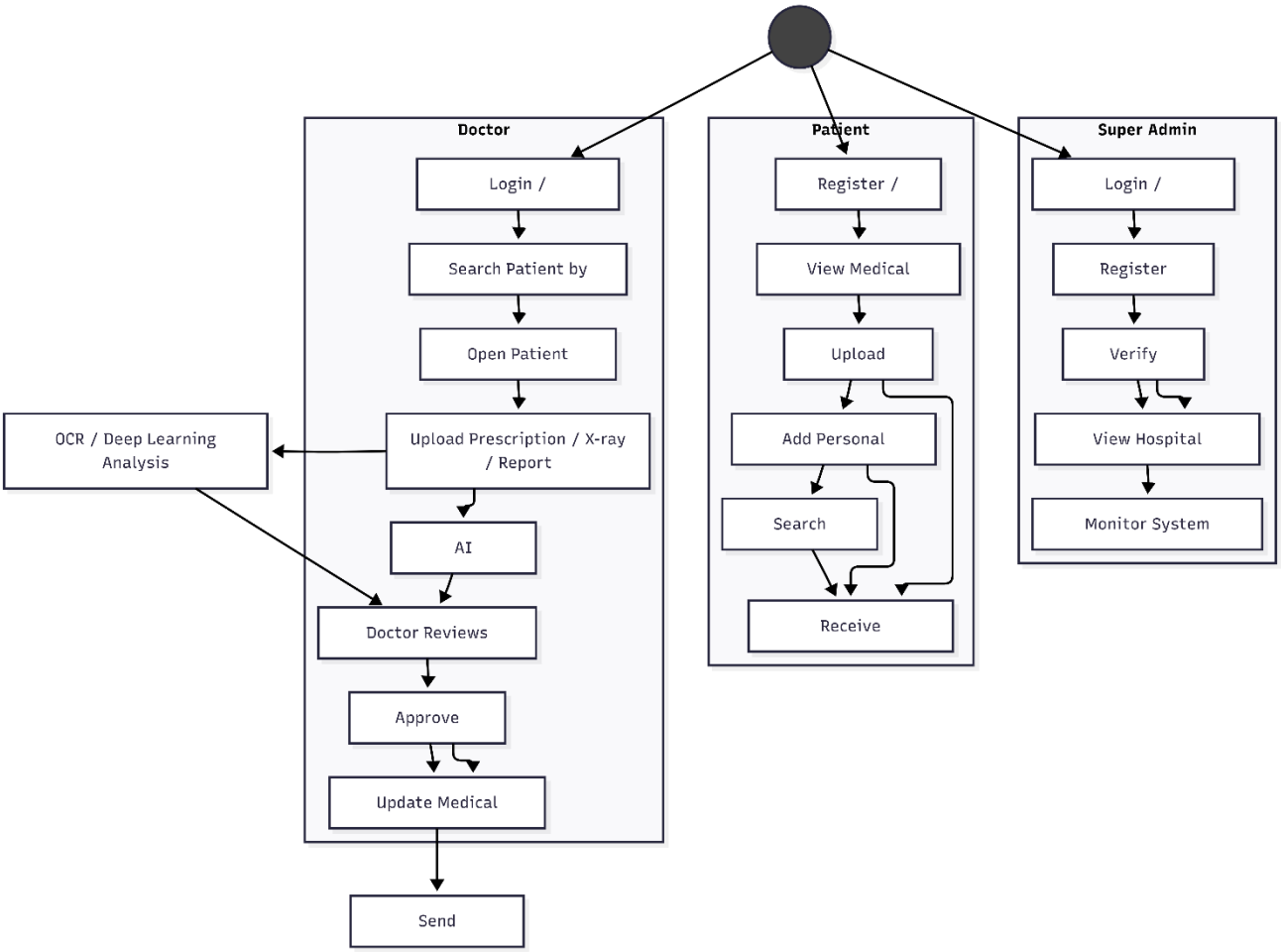


Figure 4. 3: Process Flow Representation of Medrotic

4.4 Design Models

The design model of Medrotic presents the static and dynamic structure of the system using UML diagrams. Since Medrotic is an object-oriented, role-based, AI-integrated healthcare application, the most appropriate design artifacts are the Class Diagram and Sequence Diagrams for the three

dashboards: Doctor, Patient, and Super Admin. These diagrams explain how the system is structured internally and how the major workflows are executed across different roles.

The class diagram captures the static architecture of the system, including inheritance, associations, composition, and dependencies among the main entities. The sequence diagrams show the time-based interaction between users, frontend screens, backend services, AI modules, and database operations. Together, these diagrams provide a complete view of how Medrotic operates from both structural and behavioral perspectives.

4.4.1 Class Diagram

The class diagram of Medrotic represents the core object-oriented structure of the system. At the center is the User base class, which stores shared information such as CNIC, name, email, password hash, phone number, role, and account creation time. The specialized classes Patient, Doctor, and Super Admin inherit from this base class, which reflects the role-based inheritance approach used in the system design. This structure reduces redundancy and makes it easier to manage common fields across all user types.

The Patient class is associated with multiple MedicalRecord and Document entities, because each patient may have many records and uploaded files over time. The Document entity is linked to the AIResult entity, which stores OCR output, deep learning findings, confidence score, approval status, and review metadata. The MedicalRecord entity stores diagnosis, treatment, record date, and creator role, while the Notification entity handles alerts sent for updates, approvals, and important events. The Doctor class is linked to AI approval operations, and the Super Admin class is linked to verification and hospital management workflows.

This class design is appropriate because it maps directly to the real-world healthcare domain. It supports abstraction, encapsulation, inheritance, and association in a clean way, while also matching the relational schema implemented in MS SQL Server through Entity Framework Core. The model also supports maintainability because changes in shared user behavior can be handled in one place through the base class, while role-specific logic remains separated in derived classes.

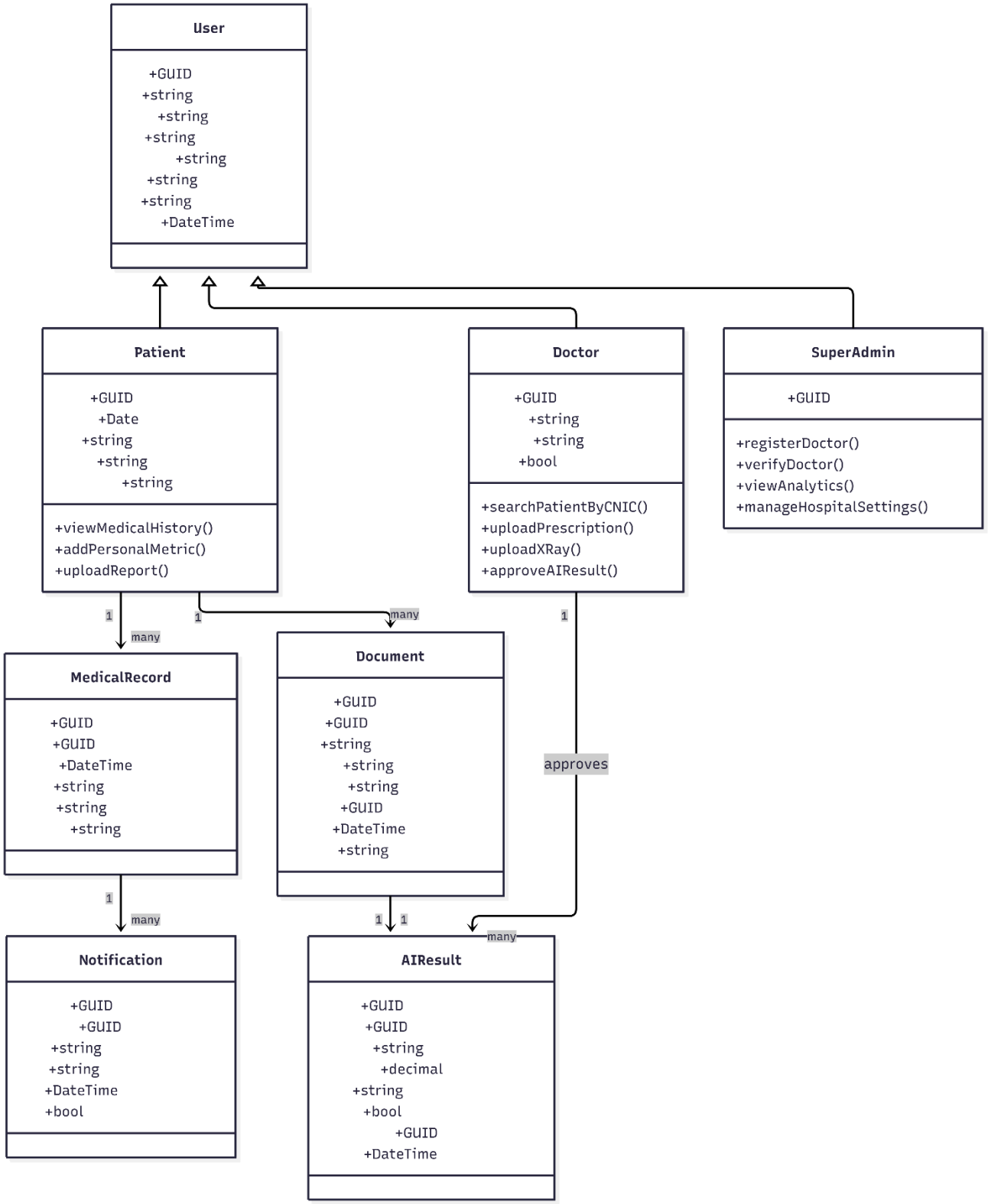


Figure 4. 4: Class Diagram of Medrotic

4.4.2 Sequence Diagram for Doctor Dashboard

The sequence diagram for the Doctor dashboard describes the workflow used when a doctor accesses patient data and uploads medical documents for AI processing. The interaction typically begins when

the doctor logs in to the system and opens the patient search screen. The doctor enters the patient's CNIC, and the backend retrieves the patient's profile and medical history from the database. This allows the doctor to review the patient's existing information before starting any new clinical action.

After accessing the record, the doctor may upload a prescription, X-ray, or test report. The uploaded file is then forwarded to the AI processing service, where OCR or deep learning analysis is performed depending on the document type. If the upload is critical, especially in the case of X-rays, the AI result is returned to the doctor for review and approval before saving it into the patient's medical record. If the doctor approves the result, the system updates the record, stores the AI output, and triggers a notification.

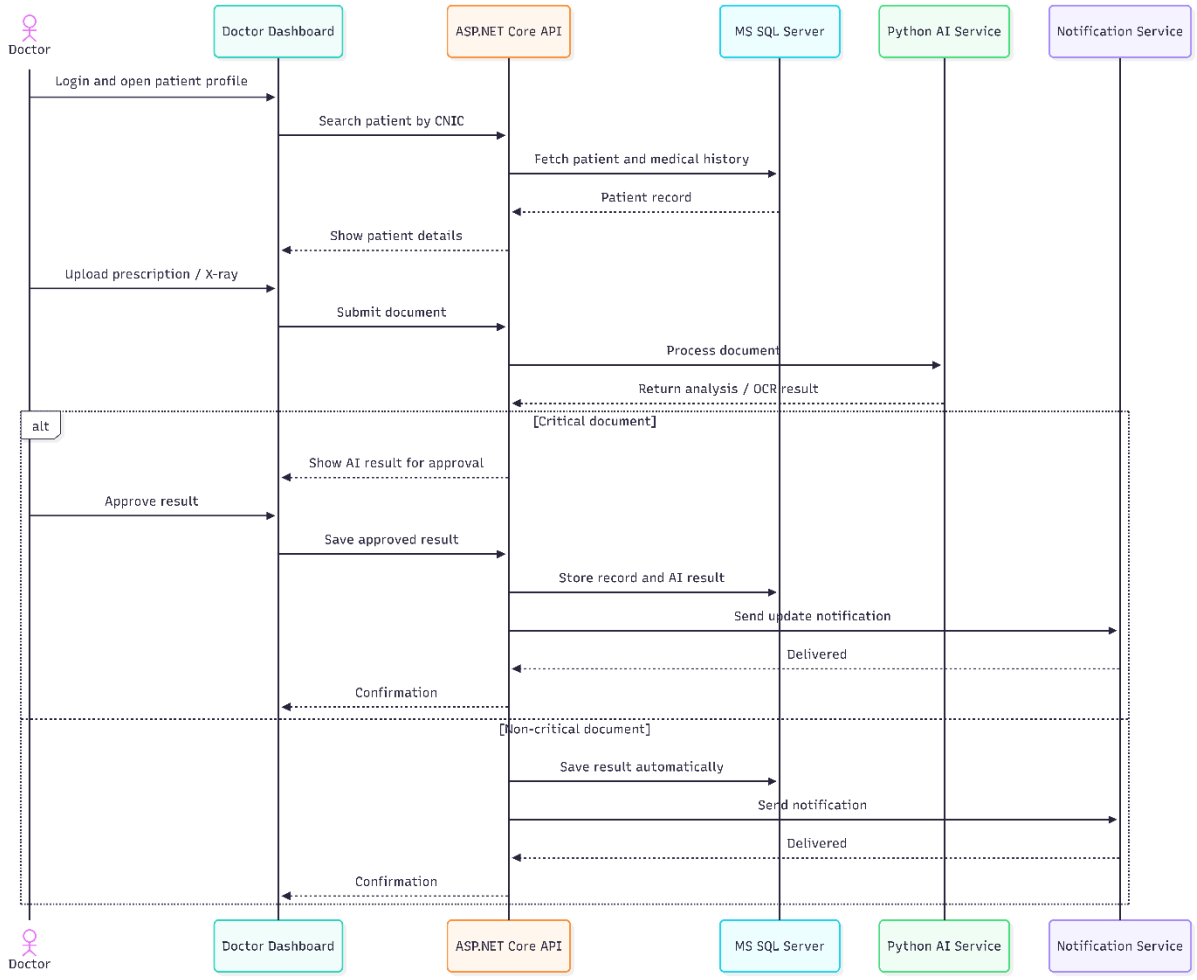


Figure 4. 5: Sequence Diagram of Doctor Dashboard

This sequence is important because it captures the most clinically sensitive part of Medrotic. Doctors are not passive users; they actively supervise AI-generated insights and decide whether a result should become part of the permanent record. The diagram therefore shows a balanced interaction between automation and medical oversight, which is a central design principle of the project.

4.4.3 Sequence Diagram for Patient Dashboard

The sequence diagram for the Patient dashboard represents the self-service flow available to patients through Medrotic. The process usually begins when the patient logs in or completes self-registration using CNIC. Once authenticated, the patient can access the dashboard and perform actions such as viewing medical history, uploading a test report, adding personal health metrics, searching for doctors, or interacting with the chatbot. The system responds by retrieving or updating the relevant patient records from the database.

When the patient uploads a non-critical report, the file is sent to the AI module for OCR or report analysis. In such cases, the system may auto-process the file and save the result directly if no manual approval is required. If the patient adds personal metrics such as blood pressure or sugar level, the system stores those values in the history timeline as patient-generated health data. Similarly, when the patient uses the chatbot, the system forwards the query to the AI service and displays a contextual response based on available data and rules.

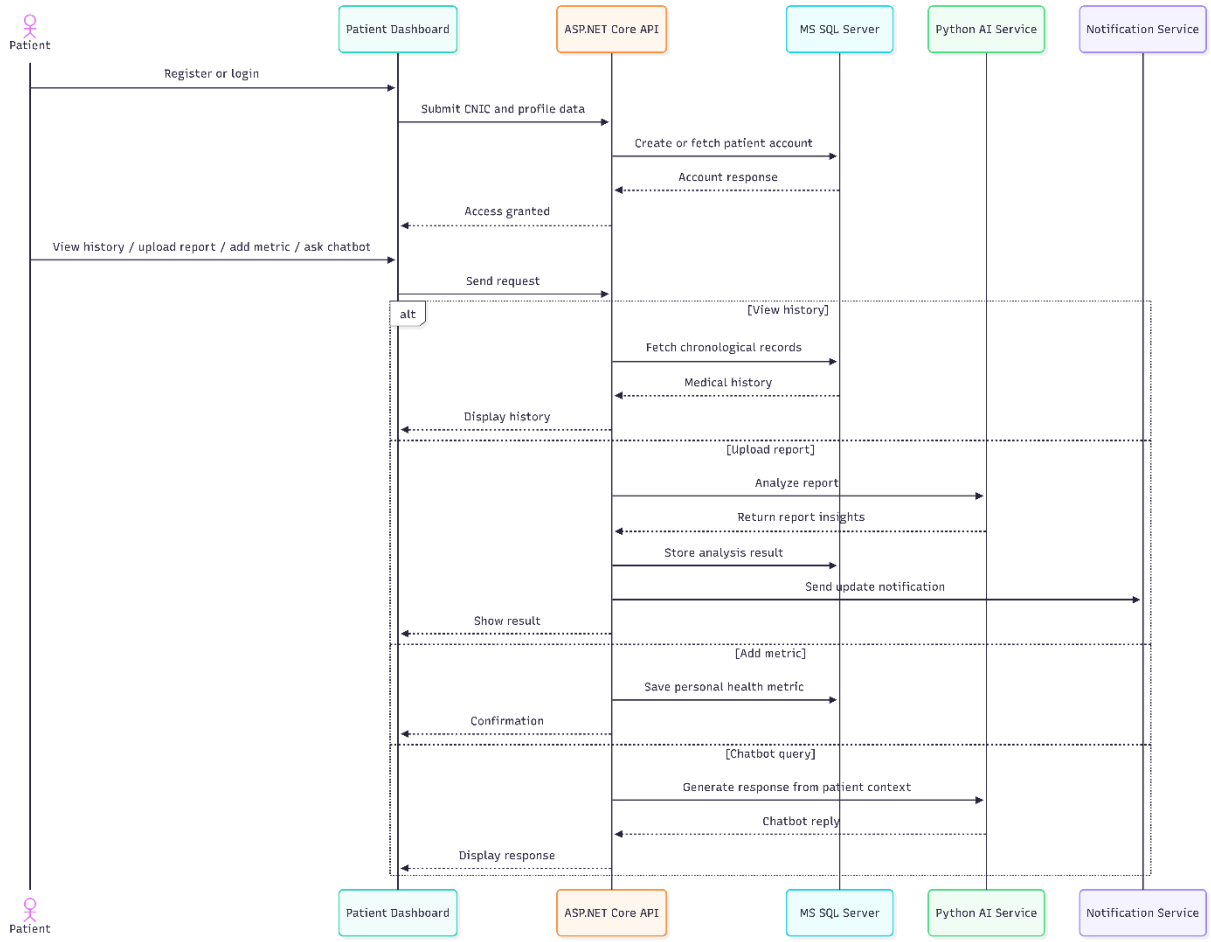


Figure 4. 6: Sequence Diagram of Patient Dashboard

4.4.4 Sequence Diagram for Admin Dashboard

The sequence diagram for the Super Admin dashboard shows the administrative workflows that keep the system properly managed. The Super Admin first logs into the admin panel and accesses the control dashboard. From there, the admin can register new doctors, verify doctor accounts, view institutional analytics, and monitor system-level activity. The dashboard retrieves the relevant information from the backend and database, then displays it in a structured and real-time format.

When a doctor is registered, the system stores the new user information and creates the associated doctor profile. In case of verification, the admin updates the doctor's status so that the doctor can begin using the clinical functions of the platform. For analytics, the system collects counts and trends related to patients, doctors, medical records, and usage patterns, then presents them on the admin dashboard.

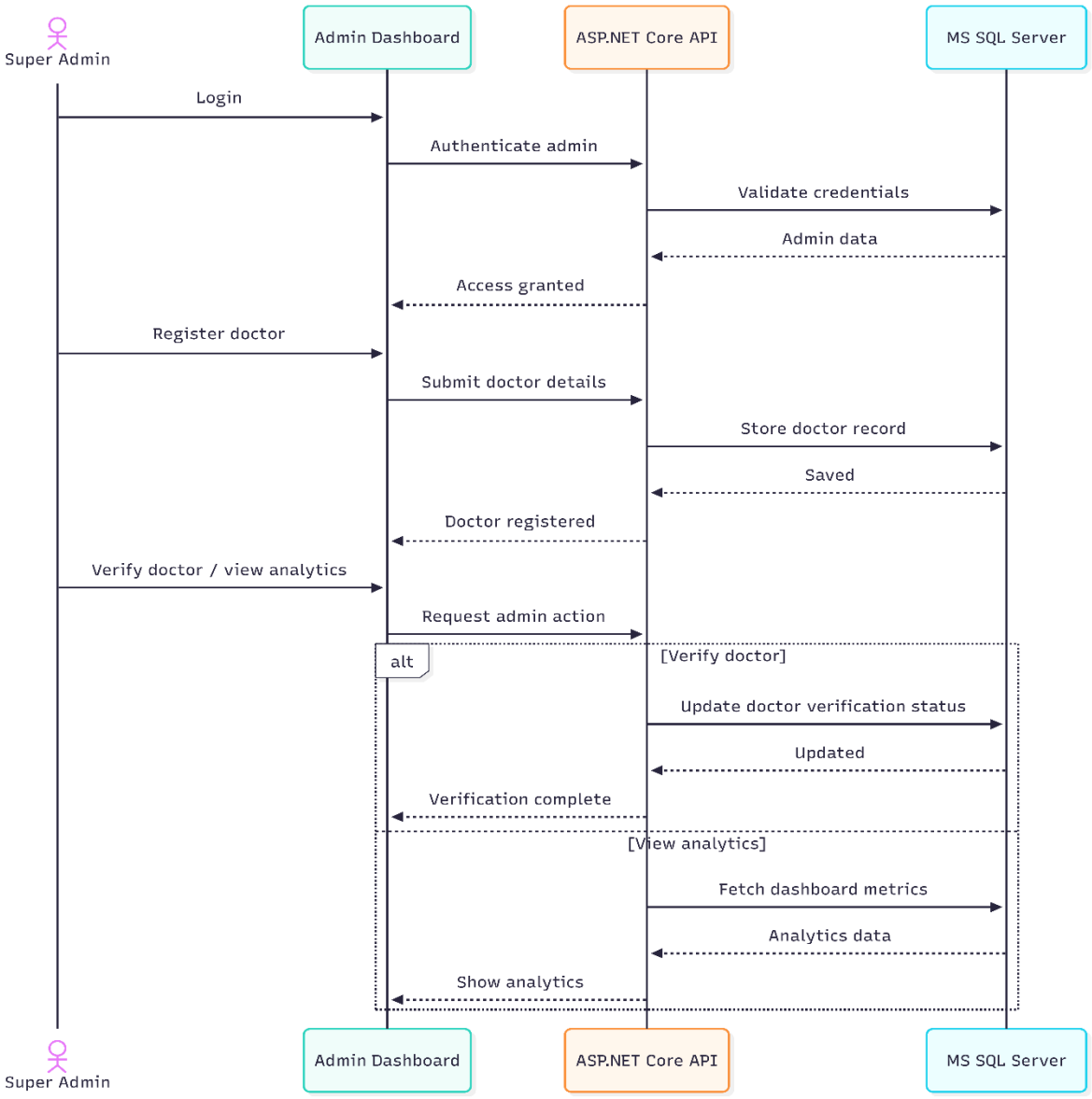


Figure 4. 7: Sequence Diagram of Admin Dashboard

Chapter 5
Implementation

5 Implementation

The implementation of Medrotic translates the approved requirements and design models into a functioning software product. It combines frontend development, backend service creation, AI integration, database connectivity, and communication features into one coordinated system. The implementation follows the architecture described in the SDD and reflects the iterative development strategy selected during project planning. In practice, this means the system was built in modules so that each major feature could be developed, connected, and tested without affecting the entire application at once.

The implementation phase focuses on making the system usable in real workflows. Frontend screens were connected to backend endpoints, AI services were integrated for OCR and deep learning tasks, and database operations were tied to the CNIC-based patient structure used throughout the platform. Notification and real-time features were also added where needed so that the platform could support practical healthcare interactions rather than remaining only a static academic prototype.

5.1 Algorithm

Several core algorithms power the working of Medrotic. The patient identification algorithm validates CNIC and retrieves patient data without duplication. The document classification algorithm decides whether an uploaded file should pass through OCR, deep learning, or approval workflows. The AI analysis algorithm handles medical image or report processing and returns structured findings and confidence values.

The approval algorithm ensures that critical AI output remains pending until reviewed by a doctor. The medical record update algorithm then appends validated findings into a chronological patient history. Additional algorithms support notification dispatch and analytics generation. These algorithms are not isolated theoretical steps; they are embedded in the working logic of the platform and directly affect how users experience healthcare workflows.

In practical terms, the implementation algorithms are designed to maintain correctness, speed, and role-based control. For example, a doctor can trigger a workflow by uploading a prescription or X-ray, but the system decides how that input should be processed and whether human approval is required before saving. This helps keep the platform secure, structured, and clinically supervised. Automated validation checks and exception handling should be incorporated to detect errors or incomplete data before processing continues. Detailed logging and performance monitoring will further support transparency, troubleshooting, and continuous optimization of the workflows.

5.2 External APIs

Medrotic depends on both internal and external service interfaces. Python FastAPI **[10]** endpoints are used for OCR and deep learning inference, creating a clear interface between the backend application layer and AI components. The notification system uses external communication support through SMTP for email and SMS-oriented services for alerts. WebSockets are used for real-time interactivity such as chatbot updates and dashboard refresh behavior.

These APIs are important because they let the platform remain modular. Instead of placing all logic in a single tightly coupled application, Medrotic allows backend services to communicate cleanly with AI and communication layers. This improves maintainability and future extensibility while keeping the implementation aligned with service-oriented development practices.

The API layer also helps separate responsibilities clearly. The main backend handles authentication, authorization, and record management, while the AI services focus only on inference and text extraction. Similarly, communication APIs manage notifications and real-time updates without mixing those responsibilities into the core medical workflow logic.

Table 5.2. 1: Details of APIs used in the Project

Name of API	Description of API	Purpose of usage	Function/class in which it is used
FastAPI (Python)	AI microservice endpoints	OCR, X-ray, report analysis	router.py (/ocr/prescription, /reports/analyze, /xray/analyze)
AIML API (aimlapi.com)	Multi-modal AI service	OCR + report extraction	aimlapi_client.py
Twilio	SMS notification service	Real-time alerts	AuthController.cs
SMTP	Email service	Notification fallback	AuthController.cs
FastAPI Chatbot	RAG chatbot inference	Patient queries	router.py
RAG History API	Patient history for AI	RAG context retrieval	PatientsController.cs, backend_client.py
Metrics Store	Local health data	Patient metrics	PatientController.cs

Name of API	Description of API	Purpose of usage	Function/class in which it is used
	storage		
System Logs API	Internal monitoring	Admin logs	AdminController.cs
Hugging Face Hub	Model loading (transformers)	Medical model inference	llm_loader.py

5.3 User Interface

The user interface of Medrotic is designed to be role-specific, responsive, and easy to navigate. The Super Admin portal focuses on monitoring and management, providing doctor registration capabilities and institutional analytics. The Doctor portal emphasizes clinical efficiency through CNIC-based patient access, upload workflows, and AI result review. The Patient portal prioritizes transparency and convenience by allowing self-registration, history viewing, report upload, metric entry, chatbot use, and doctor search **[8]**.

The interface follows a modern dashboard-based layout with separate pages for authentication, public information, patient services, doctor workflows, and administrative control. Public-facing pages such as Home, About, Contact, FAQ, Login, and Signup introduce the platform and help users understand the purpose of Medrotic. Once signed in, users are directed to role-specific dashboards where the available options are limited to their permissions and responsibilities. This structure improves usability and supports secure, organized interaction with the platform.

5.3.1 Login Page

The Login Page is the first authenticated entry point of Medrotic. It allows registered users to access the system by entering valid credentials and routes them to the correct dashboard based on their role. The page is kept simple and focused so that users can log in quickly without distraction, which is important for both medical staff and patients who need fast access to the system.

Multi-factor authentication should be supported to enhance account security and protect sensitive health information. The interface must provide clear feedback for login errors and password recovery options to reduce user frustration. Accessibility best practices should be followed to ensure the page is usable across devices and for users with varying needs.

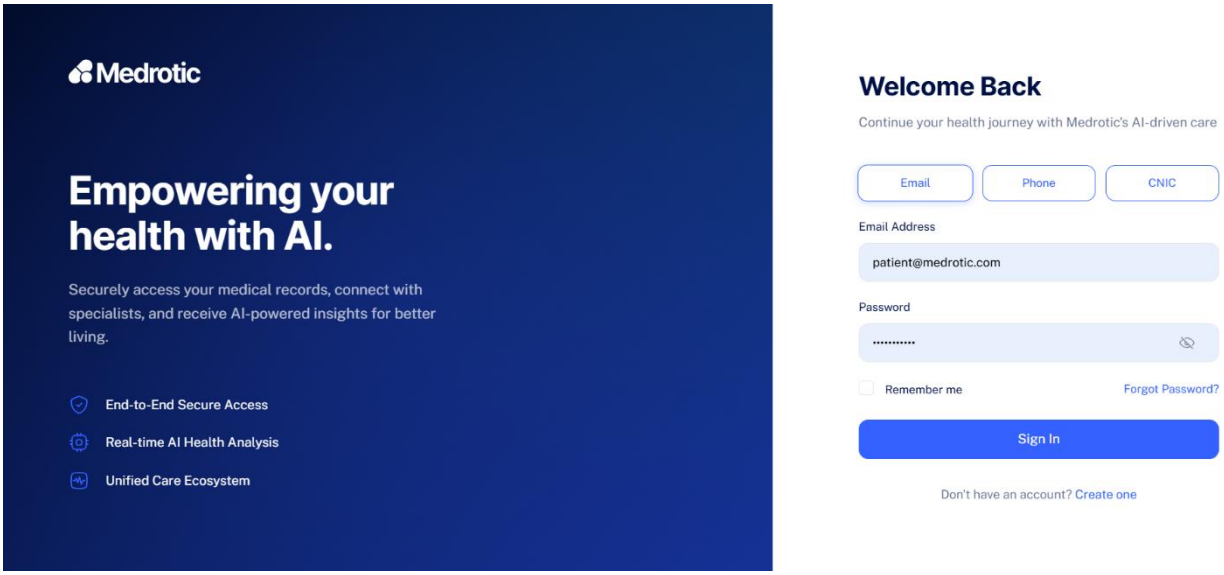


Figure 5. 1: Login Page

5.3.2 Signup Page

The Signup Page is used by new users to create an account in the system. It collects essential information required to create a secure user profile and connect the user to the appropriate role. For patient users, it helps in onboarding and registering their health identity using CNIC as the main unique identifier.

The page is designed to be straightforward and easy to complete. Input fields are arranged in a logical order so that users can enter information without unnecessary confusion. This page is especially important because it expands accessibility and allows new patients to join the system independently.

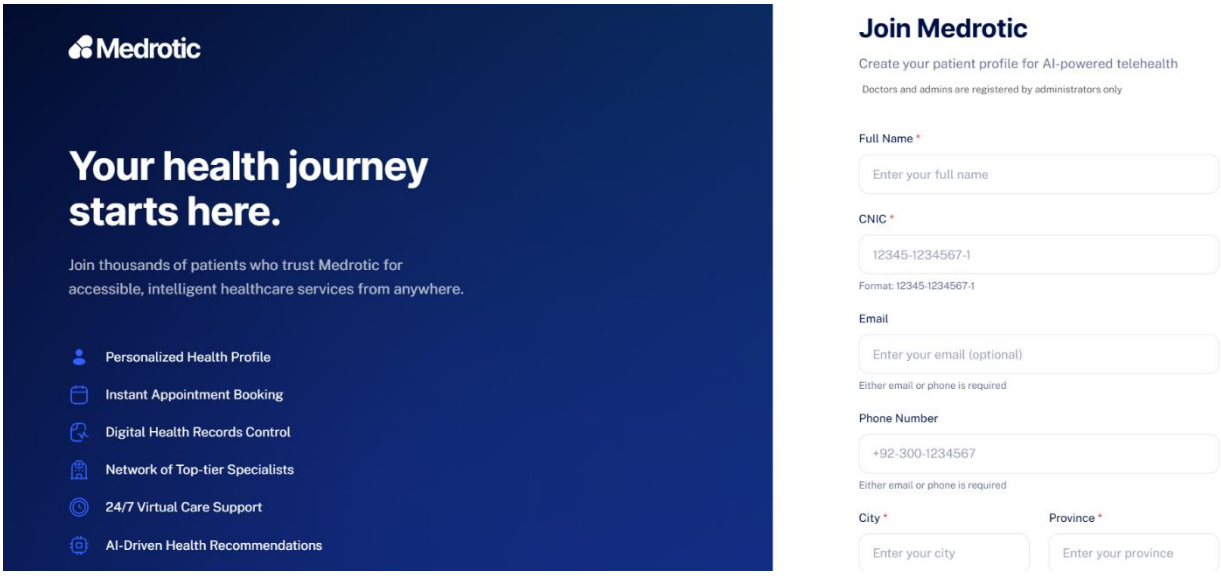


Figure 5. 2: Signup Page

5.3.3 About Page

The About Page introduces Medrotic and explains its purpose, features, and overall mission. It gives visitors a brief understanding of how the platform improves healthcare operations through digital record management, AI support, and role-based dashboards. This page helps establish the identity and professionalism of the system.

The page is useful for new users, committee members, and evaluators because it summarizes the project in a clear and readable form. It also helps users understand how Medrotic fits into the healthcare context before they interact with the actual system functions.

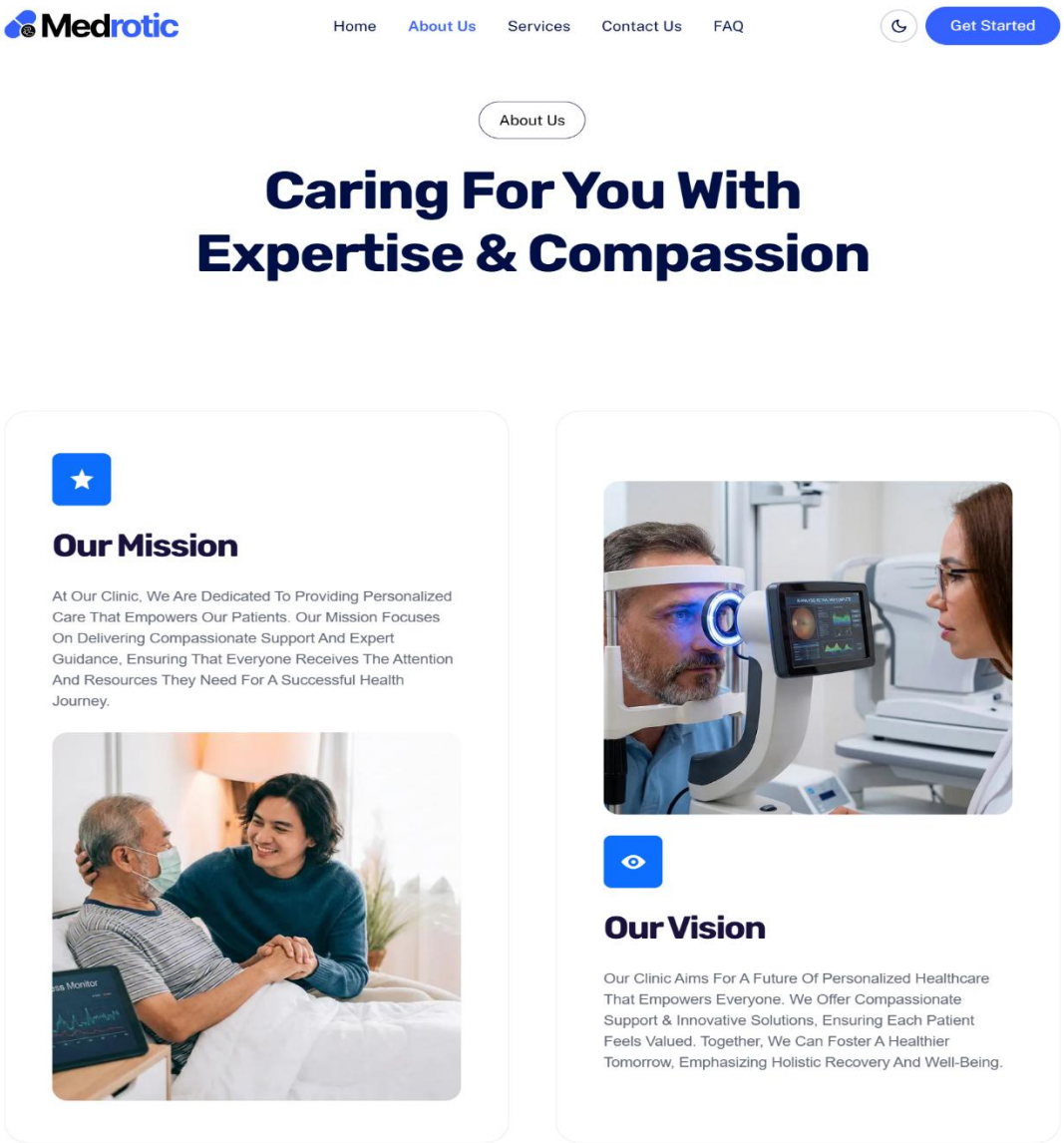


Figure 5. 3: About Page

5.3.4 Contact Page

The Contact Page provides a communication channel for users who need support, want to ask questions, or want to send feedback. It usually contains a contact form, email details, or other communication information that allows users to reach the project or system administrators.

This page improves accessibility and trust because users know there is a way to request help if they face any issue. In a healthcare-related system, this kind of support page is important for both technical and user confidence reasons.

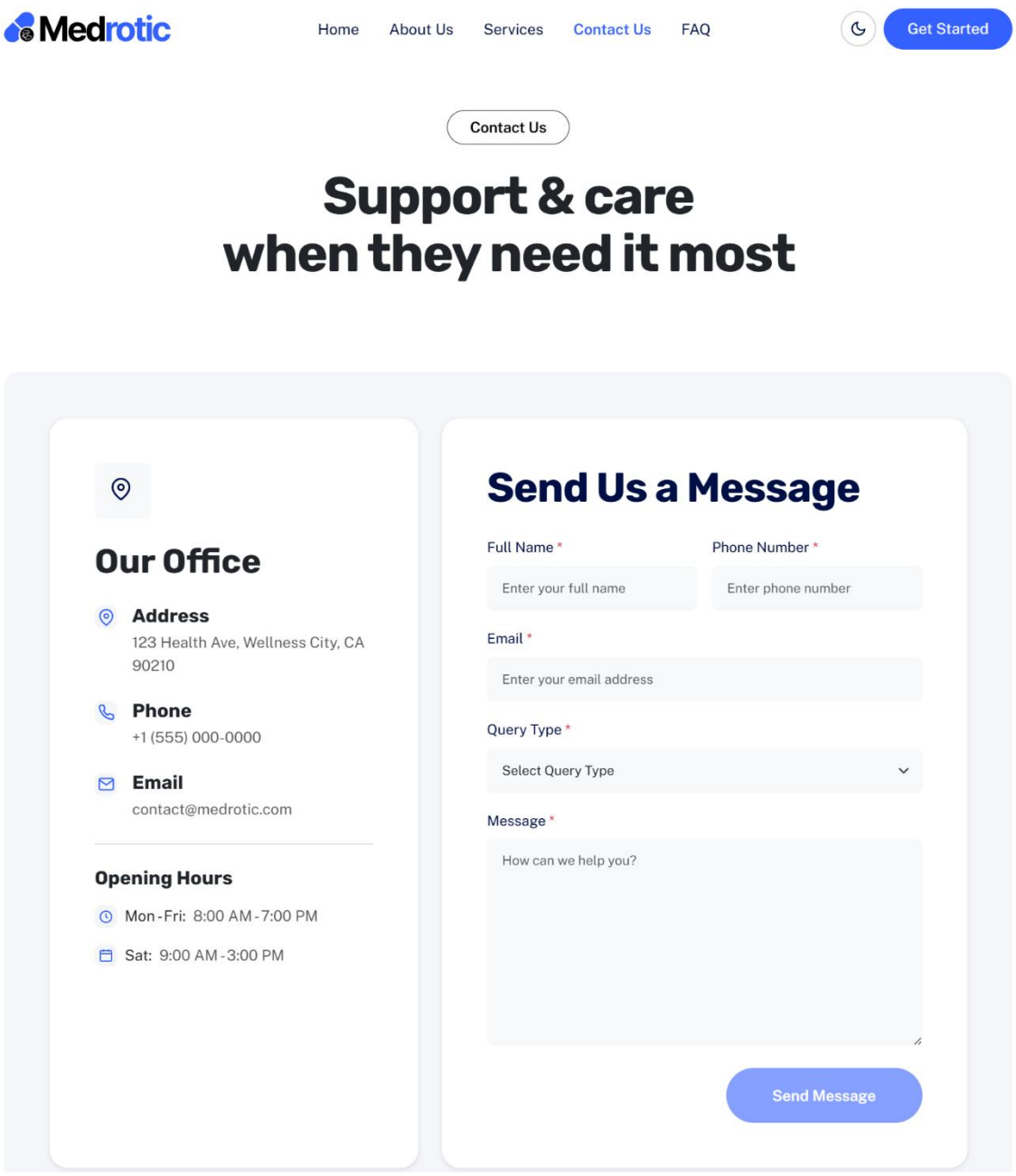


Figure 5. 4: Contact Page

5.3.5 Home Page

The Home Page serves as the landing page of Medrotic and presents the platform in a welcoming and structured way. It introduces the core purpose of the system, highlights important features, and directs users toward login or other public pages. The page acts as the main entry point for visitors and provides a first overview of the application.

This page is important because it presents the project professionally and gives users a quick understanding of what Medrotic offers. The home page also plays a role in navigation by connecting users to major sections of the system and setting the visual tone for the entire platform.

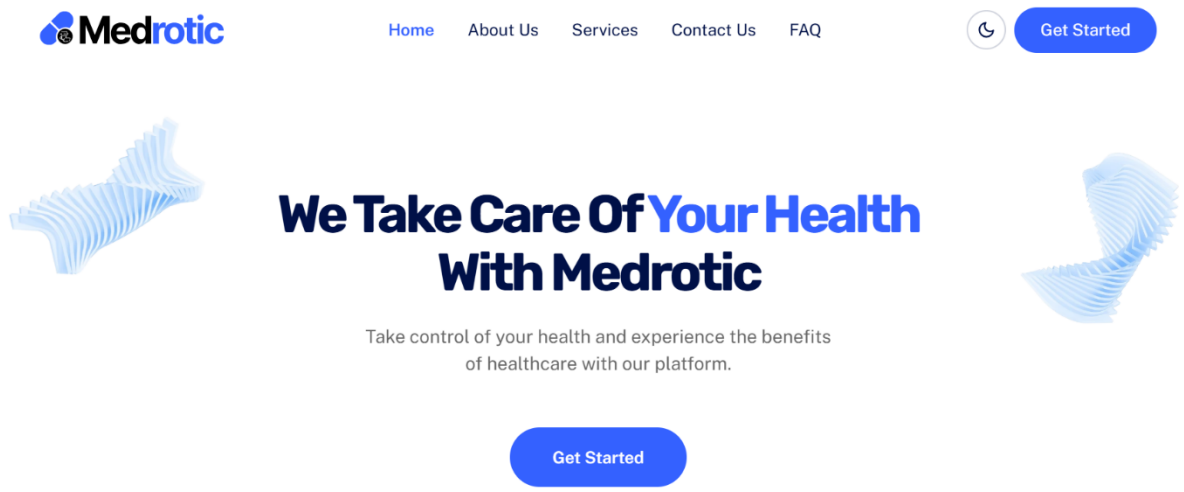


Figure 5. 5: Home Page

5.3.6 FAQ Page

The FAQ Page answers common questions related to the platform, its features, and its usage. It is designed to reduce confusion and support users who need quick clarification without contacting support directly. The page helps make the system easier to use by providing guidance in a structured format.

For a healthcare platform, FAQ content is especially helpful because users may need support related to registration, login, document upload, dashboard access, or general usage. This page improves self-service and reduces repetitive support requests.

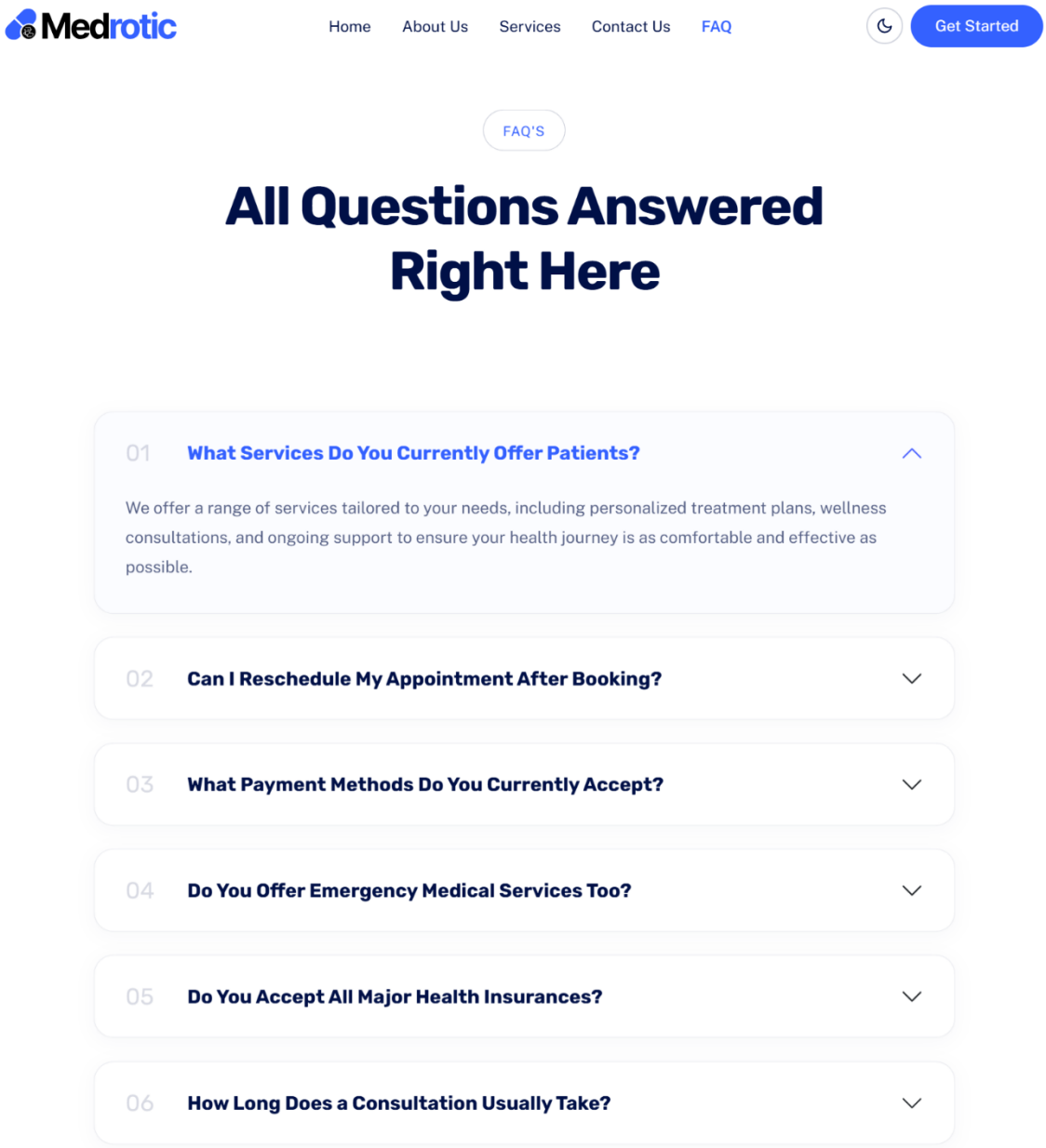


Figure 5. 6: FAQs Page

5.3.7 Doctor Dashboard

The Doctor Dashboard provides a centralized workspace where the doctor can manage patient-related tasks, access records, and handle clinical workflows efficiently in one place. It should present key patient insights, recent activity, and pending tasks through an intuitive and organized interface. Quick access to search, upload, and review tools must be available to minimize navigation time during busy clinical hours. The dashboard should also support customizable widgets so doctors can tailor the workspace to their workflow preferences.

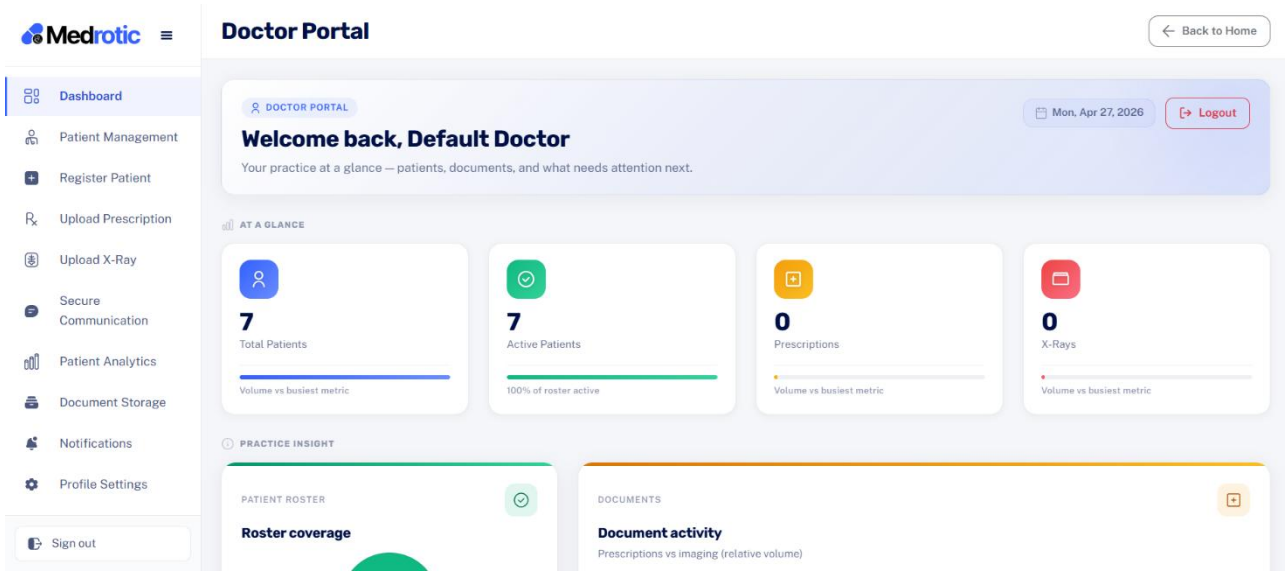


Figure 5. 7: Doctor Dashboard

5.3.8 Prescription and X-ray Upload Pages

The Prescription Upload Page allows the doctor to upload prescription images so the system can process them and store the extracted information in the patient record. It should support multiple image formats and provide instant upload status feedback.

The X-ray Upload page is used by the doctor to upload X-ray images for AI-based analysis and diagnostic support before the result is saved. The page must ensure secure transfer and display processing progress to the user.

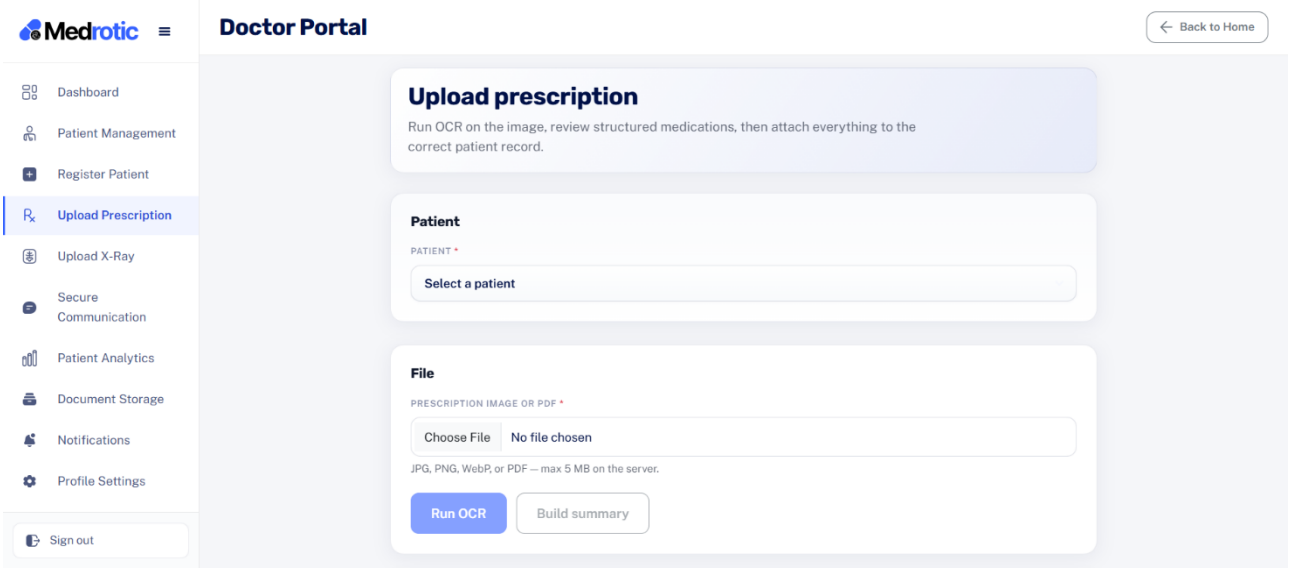


Figure 5. 8: Prescription Upload Page

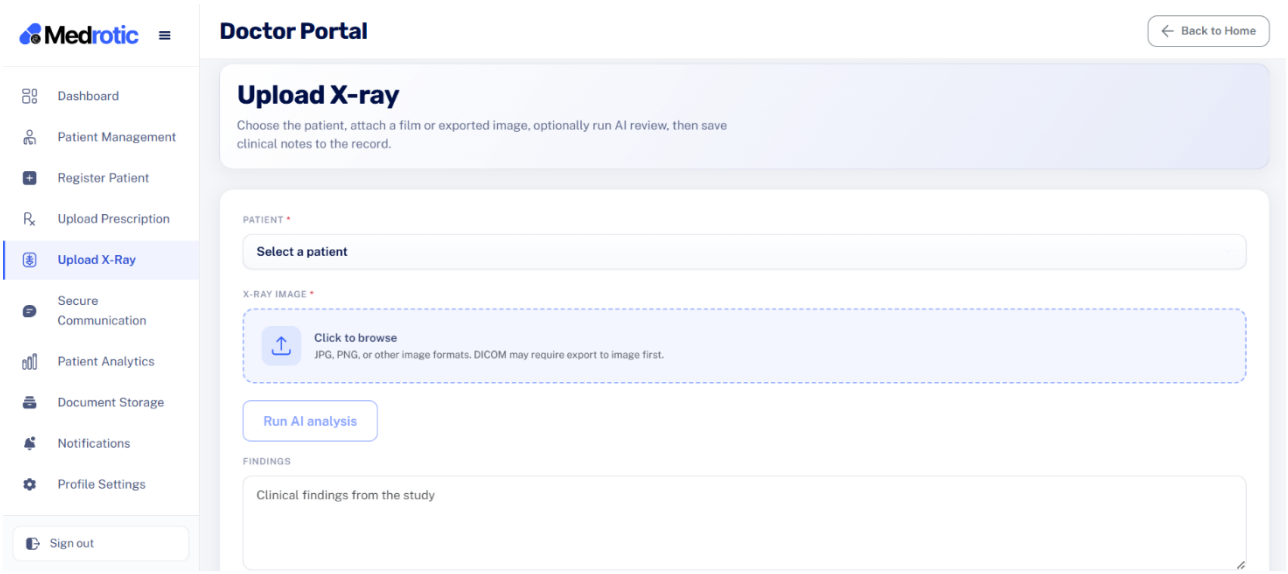


Figure 5. 9: X-ray Upload Page

5.3.9 Patient Dashboard

The Patient Dashboard gives users a personal space to view their records, interact with the system, and manage health-related information easily. It should highlight recent updates, reminders, and quick access to essential health tools.

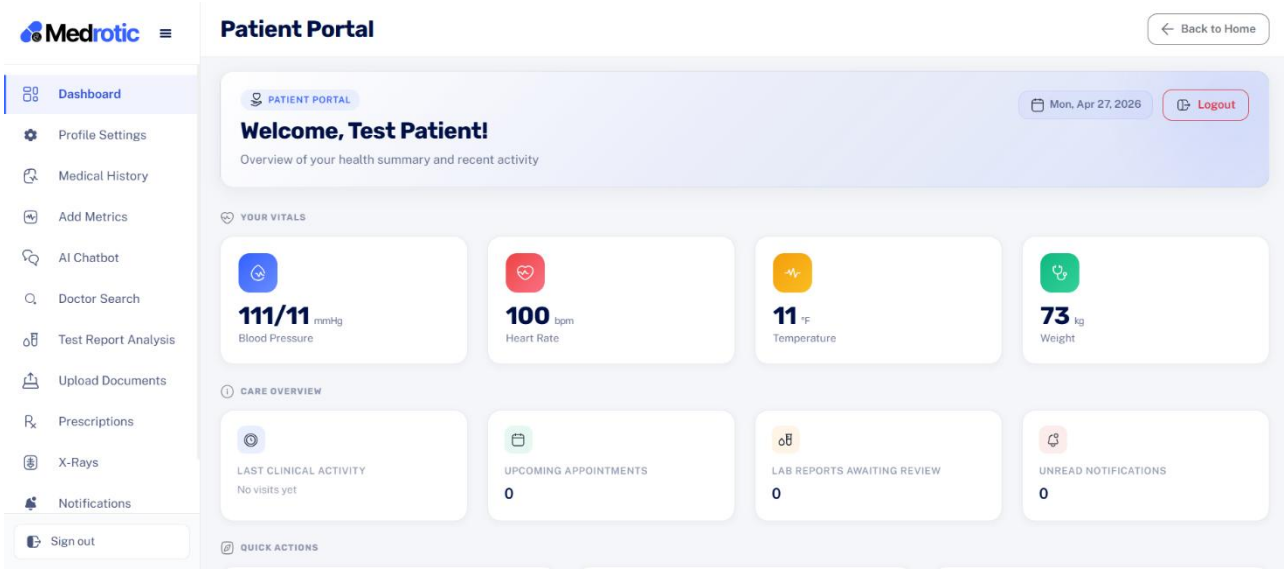


Figure 5. 10: Patient Dashboard

5.3.10 Personal Metrics Page

This page lets the patient enter personal health values such as blood pressure or sugar levels to maintain a self-monitored health log. It should include simple validation and visual trends to help patients track their progress over time.

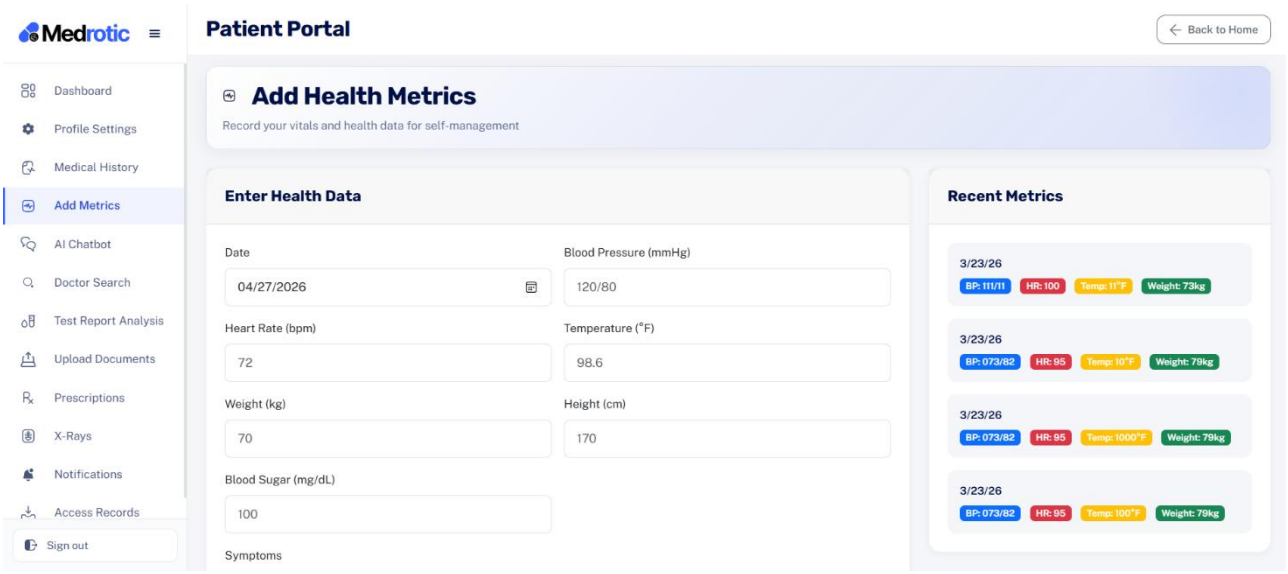


Figure 5. 11: Personal Metrics Page

5.3.11 Doctor Search Page

The Doctor Search Page helps patients find doctors by specialty, city, or other filters so they can connect with the right healthcare provider.

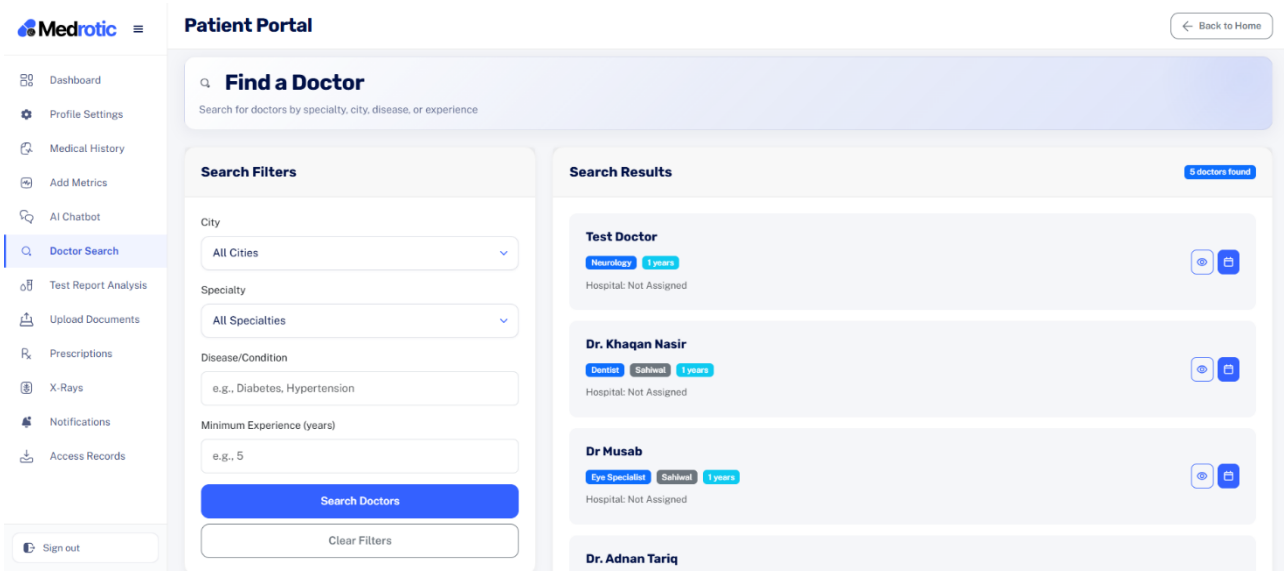


Figure 5. 12: Doctor Search Page

5.3.12 Chatbot Page

The Chatbot Page provides an interactive AI assistant that answers patient queries and offers quick guidance based on available data. It should provide conversational support for common health questions and guide users to the right features or resources. The chatbot must escalate complex or critical queries to human professionals when necessary.

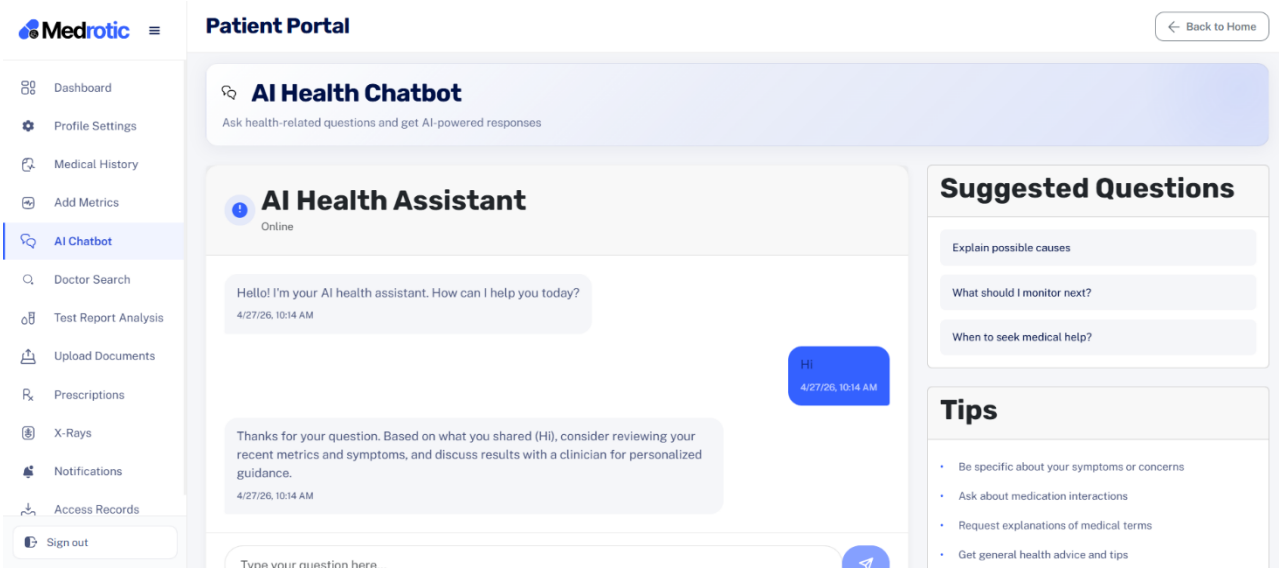


Figure 5. 13: AI Chatbot Page

5.3.13 Admin Dashboard

The Admin Dashboard gives the Super Admin a high-level overview of the system and access to management tools and analytics.

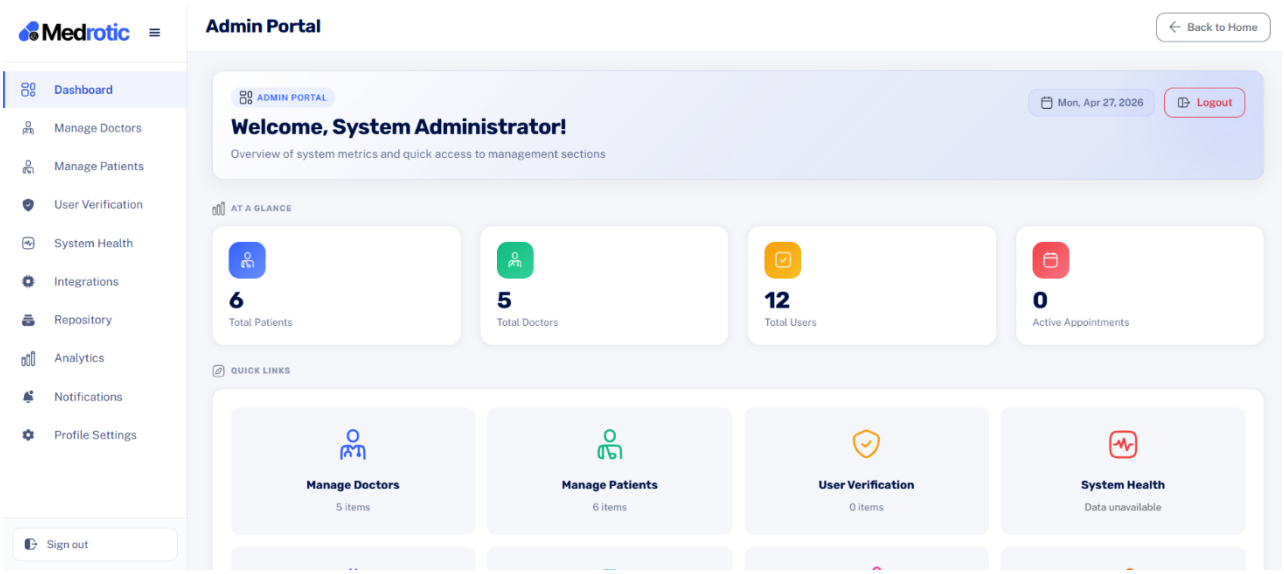


Figure 5. 14: Admin Dashboard Page

5.3.14 Doctor Management Page

This page allows the admin to add, edit, or manage doctor profiles and maintain the doctor list within the system. It should include verification workflows to ensure the accuracy of professional credentials and contact details. Bulk management and search capabilities should be provided to simplify administrative tasks.

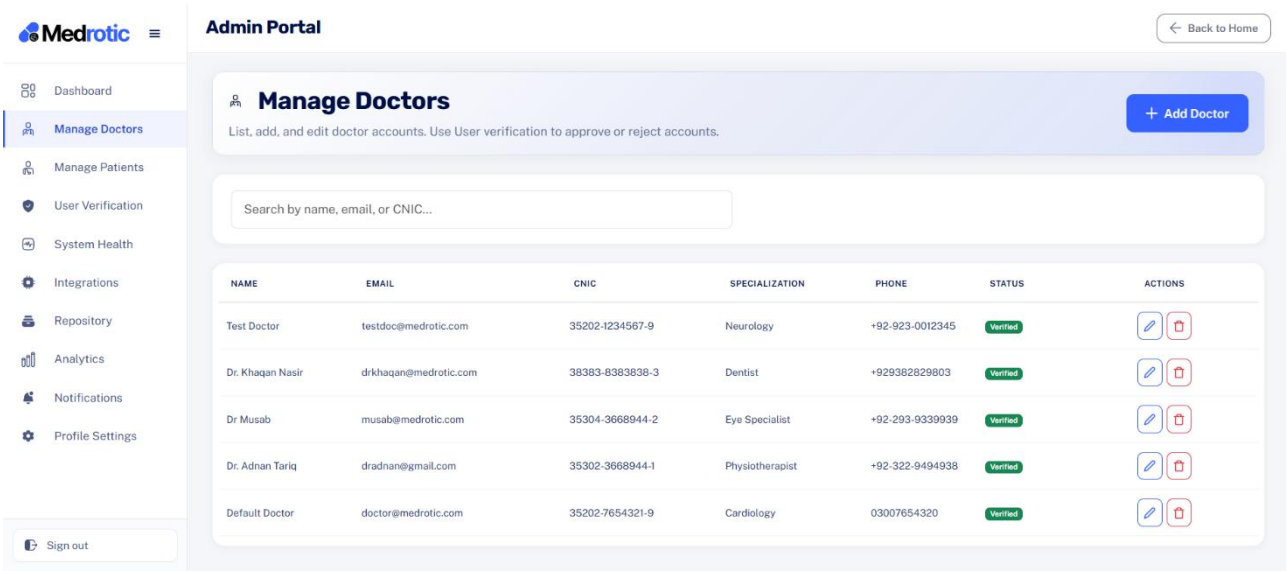


Figure 5. 15: Doctor Management Page

5.3.15 Doctor Verification Page

The Doctor Verification Page is used to review and approve new doctor accounts before they are allowed to use the platform.

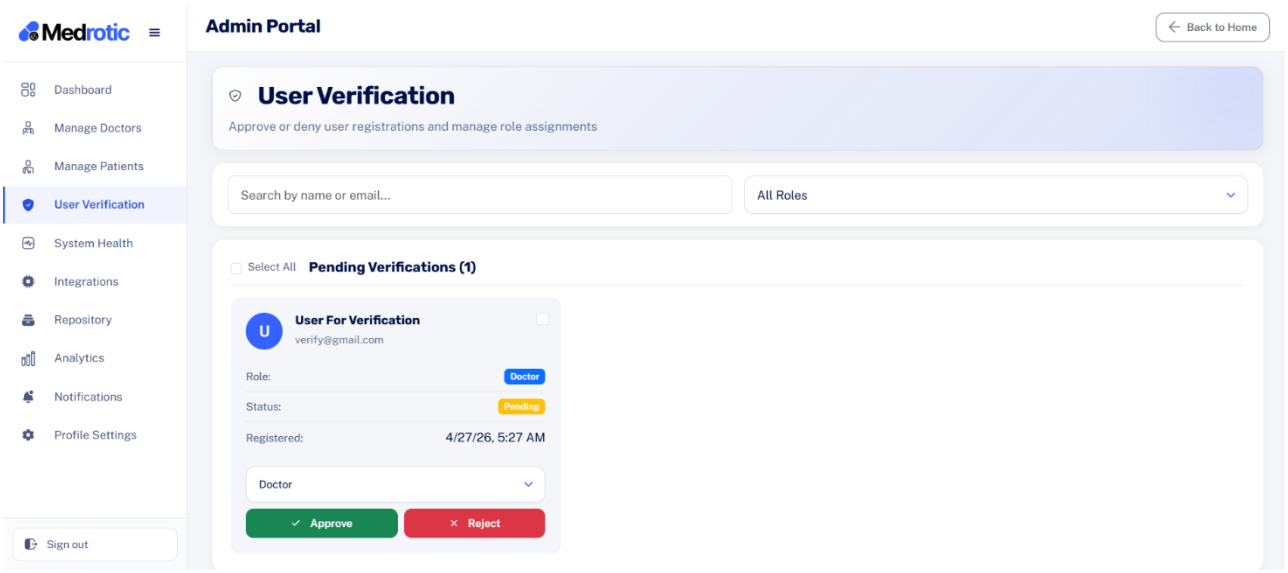


Figure 5. 16: User Verification Page

5.3.16 Analytics Page

The Analytics Page presents visual reports and key statistics that help the admin monitor the overall performance of the system. It should include interactive charts and filters to analyze trends across patients, doctors, and system usage. Real-time data updates should be supported to enable timely decision-making and operational improvements.



Figure 5. 17: Analytics Page

5.3.17 UI Design Summary

The user interface of Medrotic reflects the goals of simplicity, accessibility, and role-based organization. Public pages introduce the platform, while authenticated dashboards provide structured access to features based on user type. The consistent layout across pages helps users feel familiar with the system, and the separation of responsibilities makes it easier to use and maintain.

Overall, the UI contributes strongly to the usability of Medrotic because it connects technical functionality with a practical and human-centered presentation. The design supports both healthcare workflows and evaluator expectations by making the project easy to understand, easy to demonstrate, and suitable for real-world use.

Chapter 6
Testing and Evaluation

6 Testing and Evaluation

This section covers the testing and evaluation processes performed on the Medrotic platform to ensure all functionalities work as expected and meet the system requirements. It includes both manual and automated approaches to verify modules such as Doctor Registration, Patient Access via CNIC, Prescription/X-ray Upload with AI Processing, Patient Self-Service Features, and Super Admin Analytics. Comprehensive testing was conducted across all three role-based dashboards to validate end-to-end workflows from authentication to AI-driven diagnostics and notification delivery **[9]**.

6.1 Manual Testing

Manual testing involves verifying each feature of Medrotic by executing test cases without automation. The tester interacts with the system, inputs data, and observes outputs to confirm correct behavior. All user actions, doctor patient search via CNIC, document uploads with AI analysis, approval workflows, patient self-registration and chatbot interaction, and admin analytics were checked for accuracy, reliability, and role-based security.

6.1.1 System Testing

System testing in manual mode ensures that the Medrotic platform functions as a complete integrated system. Testers verified end-to-end workflows, including doctor CNIC-based patient access, prescription/X-ray upload with AI processing and approval, patient history viewing and metrics entry, chatbot queries, doctor search, and super admin analytics monitoring. Notifications, role restrictions, and AI confidence handling were also evaluated to confirm seamless performance under normal usage conditions.

Table 6. 1: System Testing Results

Test No.	Test Case	Input	Expected Result	Actual Result	Status
1	Login as Doctor	Valid username and password	Doctor dashboard opens	As expected,	Pass
2	Login as Patient	Valid patient credentials	Patient dashboard opens	As expected,	Pass
3	Register doctor by Super Admin	Doctor details, PMDC number	Doctor registered successfully	As expected,	Pass
4	Search patient by CNIC	Valid CNIC	Patient record is displayed	As expected,	Pass

Test No.	Test Case	Input	Expected Result	Actual Result	Status
5	Upload prescription for OCR	Prescription image	Text extracted and saved	As expected,	Pass
6	Upload X-ray for analysis	X-ray image	AI result generated	As expected,	Pass
7	Approve AI result	Critical AI output	Result approved and stored	As expected,	Pass
8	View medical history	Logged-in patient account	Chronological history appears	As expected,	Pass
9	Use AI chatbot	Natural language question	Relevant response generated	As expected,	Pass
10	View analytics as Super Admin	Dashboard access	Real-time metrics displayed	As expected,	Pass
11	Send notification	Triggered medical event	Notification delivered	As expected,	Pass
12	Add personal metrics	BP or sugar value	Metric stored in history	As expected,	Pass

6.1.2 Unit Testing

Unit testing focuses on individual components of the Medrotic system, such as CNIC validation, document upload handlers, AI service calls, approval logic, and notification dispatch. Each module was tested separately to ensure correct outputs for given inputs **[10]**.

Unit Testing 1: Doctor Login and CNIC Patient Search

Testing Objective: To verify that the doctor login system functions correctly and CNIC-based patient search retrieves accurate records. The test ensures valid doctors access their dashboard and can quickly locate patients, while invalid credentials or CNIC formats are properly rejected.

Table 6. 2: Doctor Login and Patient Search

No.	Test Case ID	Test Case Name	Attribute and Value Tested	Expected Result	Actual Result
1	UT-DOC-001	Doctor Login (Valid)	Username: doc123, Password: Pass123!	"Doctor logged in successfully!"	Pass
2	UT-DOC-002	Doctor Login (Invalid)	Username: wrongdoc, Password: wrong	"Invalid credentials"	Pass

No.	Test Case ID	Test Case Name	Attribute and Value Tested	Expected Result	Actual Result
3	UT-DOC-003	Patient Search (Valid CNIC)	CNIC: 35202-1234567-1	Patient record displayed	Pass
4	UT-DOC-004	Patient Search (Invalid CNIC)	CNIC: 123	"Patient not found"	Pass
5	UT-DOC-005	Empty CNIC Search	CNIC: `` | "Enter valid CNIC" | Pass |		

Unit Testing 2: Prescription Upload and OCR

Testing Objective: To ensure the prescription upload form and OCR processing work correctly. The testing verifies supported image formats are processed, extracted text is accurate, and invalid uploads are handled gracefully.

Table 6. 3: Prescription Upload and OCR

No.	Test Case ID	Test Case Name	Attribute and Value Tested	Expected Result	Actual Result
1	UT-DOC-006	Valid Prescription Upload	PNG image < 5MB	OCR text extracted and saved	Pass
2	UT-DOC-007	Invalid Image Format	TXT file	"Unsupported format"	Pass
3	UT-DOC-008	Oversized Image	10MB JPEG	"File too large"	Pass
4	UT-DOC-009	Empty Upload	No file selected	"Select prescription image"	Pass

Unit Testing 3: Patient Self-Registration

Testing Objective: To confirm patient self-registration validates CNIC uniqueness and creates complete profiles. Invalid CNIC formats and duplicates are rejected appropriately.

Table 6. 4:Patient Self-Registration

No.	Test Case ID	Test Case Name	Attribute and Value Tested	Expected Result	Actual Result
1	UT-PAT-001	Valid Registration	CNIC: 35202-9876543-2, Valid details	"Registration successful"	Pass
2	UT-PAT-002	Duplicate CNIC	Existing CNIC	"CNIC already registered"	Pass
3	UT-PAT-003	Invalid CNIC Format	CNIC: 12345	"Invalid CNIC format"	Pass
4	UT-PAT-004	Empty Fields	Missing email	"All fields required"	Pass

6.1.3 Functional Testing

Functional testing verified that the implemented features matched the functional requirements listed in the SRS. Each core requirement such as patient registration, CNIC-based lookup, prescription OCR, X-ray analysis, chatbot response, and analytics display were tested against the expected behavior. The results confirmed that the system behaves according to the approved design.

Functional Testing 1: Doctor Module

Objective: Verify doctor workflow from login to AI-assisted diagnosis and record update.

Table 6. 5: Doctor Module Functional Testing

No.	Test Case ID	Test Case Name	Attribute and Value Tested	Expected Result	Actual Result
1	FT-DOC-001	CNIC Patient Search + X-ray Upload	CNIC: 35202-1234567-1, X-ray PNG	AI analysis generated, pending approval	Pass
2	FT-DOC-002	AI Result Approval	Approve pending X-ray result	Result saved to history	Pass
3	FT-	Prescription OCR	Upload prescription	Text extracted	Pass

No.	Test Case ID	Test Case Name	Attribute and Value Tested	Expected Result	Actual Result
	DOC-003		image	automatically	
4	FT-DOC-004	Role Restriction Test	Patient attempts approval	Access denied	Pass

Functional Testing 2: Patient Module

Objective: Ensure patient self-service features work seamlessly.

Table 6. 6: Patient Module Functional Testing

No.	Test Case ID	Test Case Name	Attribute and Value Tested	Expected Result	Actual Result
1	FT-PAT-001	Self-Registration + History View	New CNIC + Register	History view shows registration	Pass
2	FT-PAT-002	Add Personal Metrics	BP: 120/80, Sugar: 95	Metrics added to history	Pass
3	FT-PAT-003	AI Chatbot Query	"What is my BP?"	Shows latest BP reading	Pass
4	FT-PAT-004	Doctor Search	Specialty: "Cardiology"	List of cardiologists	Pass

Functional Testing 3: Super Admin Module

Objective: Validate administrative oversight and analytics.

Table 6. 7: Super Admin Module Functional Testing

No.	Test Case ID	Test Case Name	Attribute and Value Tested	Expected Result	Actual Result
1	FT-ADMIN-001	Doctor Registration	New doctor details	Doctor added (pending verification)	Pass
2	FT-ADMIN-002	View Analytics	Dashboard access	Real-time KPIs displayed	Pass
3	FT-ADMIN-003	Role Restriction	Patient access admin	Access denied	Pass

6.1.4 Integration Testing

Integration testing was used to confirm that different parts of the system work together correctly. The frontend, backend, AI services, database, and notification modules were tested in connected workflows. For example, when a doctor uploads a document, the system sends it to the AI service, receives a result, stores the output, and sends a notification where required.

Table 6. 8: Integration Testing Results

No.	Test Case ID	Test Case Name	Integrated Modules	Expected Result	Actual Result
1	IT-001	Frontend → Backend → AI	Angular upload → ASP.NET → FastAPI OCR	OCR result returned and displayed	Pass
2	IT-002	Backend → Database	AI approval → MS SQL update	Record updated in patient history	Pass
3	IT-003	Role-Based API Access	Patient calls doctor endpoint	403 Forbidden	Pass
4	IT-004	CNIC Cross-Module Search	Patient register → Doctor search	Single unified record	Pass

6.2 Automated Testing

Automated testing ensures consistent quality through CI/CD pipelines. xUnit (.NET) covered backend APIs and services, PyTest validated Python AI modules (OCR, deep learning), and GitHub Actions executed full regression suites on every commit.

Table 6. 9: Automated Testing Tools and Coverage

Tool Name	Tool Description	Applied On	Results
xUnit	.NET unit/integration tests	Backend APIs, services	92% coverage, 187 tests
PyTest	Python AI validation	OCR, Deep Learning modules	88% coverage, 43 tests
GitHub Actions	CI/CD pipeline	All FRs/NFRs	100% pass rate

Chapter 7
Conclusion and Future Work

7 Conclusion and Future Work

This chapter concludes the project and highlights future work for Medrotic. It summarizes the overall achievements and the value delivered through the implemented system. It also reflects on the challenges faced during development and how they were addressed to meet project objectives. Finally, it outlines potential enhancements and long-term improvements to further strengthen the platform's capabilities and scalability.

7.1 Conclusion

Medrotic has been designed to revolutionize healthcare operations by centralizing patient records, automating diagnostic workflows, and connecting hospitals, doctors, and patients through a secure, AI-powered platform. The main goal of this project was to create a comprehensive digital healthcare ecosystem where doctors can quickly access patient data via CNIC, upload prescriptions/X-rays for AI analysis, and maintain clinical oversight, while patients can self-register, view their medical history, add personal health metrics, interact with AI chatbots, and search for specialists, all within one unified system **[16]**.

Medrotic successfully eliminates the inefficiencies of fragmented paper-based records, manual data entry, and disconnected healthcare tools. Traditionally, doctors spent valuable time reconstructing patient histories from scattered files, patients had limited access to their own medical information, and administrators lacked real-time institutional insights. Now, Medrotic provides instant CNIC-based patient lookup, OCR-powered prescription processing, deep learning X-ray analysis with doctor approval workflows, chronological medical history viewing, and hospital-level analytics dashboards.

The platform's role-based architecture ensures optimal usability across all stakeholders. Doctors benefit from streamlined clinical workflows with AI assistance but full professional control, patients gain unprecedented transparency and self-service capabilities, and Super Admins achieve comprehensive governance through doctor verification and performance monitoring. The intuitive Angular.js dashboards require no specialized training, making the system immediately accessible to healthcare professionals and patients alike.

From requirement analysis through rigorous testing, Medrotic's quality has been validated across all 12 functional requirements (FR-1 to FR-12) and 8 non-functional requirements. Core modules including doctor registration/verification, patient CNIC lookup with AI document processing, medical

history management, chatbot interactions, analytics dashboards, and notification systems have been thoroughly tested for both typical healthcare scenarios and edge cases (invalid CNIC formats, oversized medical images, unauthorized access attempts). All test cases passed, confirming seamless integration between ASP.NET Core backend, Python AI services, Angular frontend, and MS SQL Server data layer.

In conclusion, Medrotic fulfills its mission of delivering a production-ready, AI-enhanced healthcare management system that significantly improves clinical efficiency, patient engagement, administrative oversight, and overall healthcare delivery. It represents a practical, scalable digital transformation solution for small-to-medium healthcare institutions, successfully demonstrating the integration of modern web technologies, relational database design, AI automation, and secure software engineering principles.

7.2 Future Work

While Medrotic successfully meets all current objectives and demonstrates production readiness, several strategic enhancements can further extend its capabilities and market applicability:

7.2.1 Telemedicine Integration

Implement real-time video consultation capabilities between doctors and patients, complete with virtual waiting rooms, session recording (with consent), prescription e-signing, and integration with the existing medical record system to automatically link consultation notes and outcomes.

7.2.2 Mobile Application Development

Develop native iOS/Android applications using React Native or Flutter, providing offline access to medical history, push notifications for appointments/results, camera integration for direct X-ray/report uploads, and biometric authentication (Face ID/fingerprint) for enhanced security.

7.2.3 Advanced AI Capabilities

Expand AI services to include **[16]**:

- Predictive analytics for patient readmission risk and disease progression
- Natural language processing for automated clinical note summarization
- Multi-modal AI combining X-ray analysis with patient vitals and historical data
- Explainable AI providing doctors with confidence scores and decision rationale

7.2.4 Pharmacy Integration and e-Prescriptions

Integrate with pharmacy management systems and implement electronic prescription workflows where approved prescriptions are directly transmitted to partnered pharmacies with QR code verification, dosage tracking, and medication adherence monitoring.

7.2.5 Multi-Language and Regional Support

Add Urdu language support alongside English, with right-to-left text rendering, localized date formats, regional hospital directories, and compliance with Pakistan-specific healthcare regulations (PMDC, DRAP standards) **[17]**.

7.2.6 Enterprise Security Enhancements

- Implement HIPAA/PHIPA-compliant encryption for data at rest and in transit
- Add multi-factor authentication (SMS, authenticator apps, biometrics)
- Deploy intrusion detection systems and comprehensive audit logging
- Enable role-based data retention policies and patient data export capabilities **[17].**

7.2.7 Advanced Analytics Dashboard

Develop institutional-grade analytics including:

- Patient demographic trends and disease pattern analysis
- Doctor performance metrics and patient satisfaction scores
- AI model accuracy tracking and continuous learning feedback loops
- Revenue cycle management and operational cost analytics

These enhancements position Medrotic to evolve from a foundational healthcare platform into a comprehensive digital health ecosystem capable of serving national-scale healthcare networks while maintaining the modularity, security, and clinical reliability demonstrated in the current version.

References

[1] M. A. A.-G. e. al., "Artificial Intelligence in Healthcare: Review and Prediction Case Studies," *Engineering,* vol. 6, no. 3, pp. 291-301, 2020.

[2] World Health Organization, "WHO guideline: recommendations on digital interventions for health system strengthening," 2020.

[3] A. E. e. al., "A guide to deep learning in healthcare," Nature Medicine," vol. 25, no. 1, pp. 24-29, 2019.

[4] I. Sommerville, Software Engineering, 10th ed, Boston, MA, USA: Pearson, 2016.

[5] B. B. a. R. Turner, Balancing Agility and Discipline: A Guide for the Perplexed, Boston, MA, USA: Addison-Wesley, 2004.

[6] M. Fowler, Patterns of Enterprise Application Architecture, Boston, MA, USA: Addison-Wesley, 2003.

[7] E. G. e. al., Design Patterns: Elements of Reusable Object-Oriented Software, Reading, MA, USA: Addison-Wesley, 1994.

[8] J. J. Garrett, The Elements of User Experience: User-Centered Design for the Web, 2nd ed, Berkeley, CA, USA: New Riders, 2011.

[9] C. S. a. T. B. G. J. Myers, The Art of Software Testing, 3rd ed, Hoboken, NJ, USA: Wiley, 2012.

[10] I. M. A. Jørgensen, "Unit Testing, in Software Testing Fundamentals, 2nd ed," [Online]. Available: https://www.softwaretestingfundamentals.com/unit-testing.html.

[11] Microsoft, "ASP.NET Core Documentation," Microsoft, [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/.

[12] Angular, "Angular Documentation," [Online]. Available: https://angular.io/docs.

[13] FastAPI, "FastAPI Documentation," [Online]. Available: https://fastapi.tiangolo.com/.

[14] R. S. P. a. B. Maxim, Software Engineering: A Practitioner's Approach, 9 ed., New York, NY, USA: McGraw-Hill Education, 2014.

[15] A. J. a. G. S. A. Sillitti, Software Engineering: Process and Tools, Springer, 2008.

[16] S. R. a. P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed, Hoboken, NJ, USA: Pearson, 2020.

[17] HIPAA Journal, "HIPAA Compliance Checklist," 2024.

[18] Pakistan Medical and Dental Council, "PMDC Registration Guidelines".